Verification from Scratch

Everything you need to present A3 confidently

Self-study primer

March 2026

Chapter 1: What is Software Verification?

Chapter goal

Understand what verification means, why it is hard, and what vocabulary (bug, safety, reachability, true/false positive) you will use constantly in the A3 talk.

1. What is a bug?

- A **bug** is a program behaviour that violates a specification or expectation
- Common concrete examples you will mention in the talk:
  - **DIV_ZERO**: expression `a / b` executes when `b == 0`
  - **INDEX_OUT_OF_BOUNDS**: access `lst[i]` when `i >= len(lst)`
  - **NULL_DEREF**: attribute access on `None`
  - **UNCAUGHT_EXCEPTION**: an exception propagates out of the program
  - **SQL_INJECTION**: user-controlled string reaches a database query unescaped
- A bug is not just "the program crashed"; it is a specific *condition* being true at a specific *location*

2. What does "verify" mean?

- **Testing** checks a finite number of concrete runs — it can find bugs, never prove absence
- **Verification** reasons about *all possible* runs (or a rigorously defined subset)
- A **verifier** either:
  1. produces a *proof* that a bug **cannot** happen under stated assumptions, or
  2. produces a *witness* (concrete input) that **demonstrates** the bug
- The A3 pipeline does both: proof artifacts silence false alarms; witnesses ground true positives
- Important: "verification" in practice usually means "verification under assumptions" — nobody claims to handle all of Python perfectly

3. Safety vs liveness

- **Safety property**: "nothing bad ever happens" — expressed as a set U of bad states
  - "The program never divides by zero" is a safety property
  - "Buffer index is always in range" is a safety property
- **Liveness property**: "something good eventually happens"
  - "The program always terminates" is a liveness property
  - "Every request is eventually answered" is a liveness property
- A3 focuses on **safety**: can we reach a bad state?
- Reason: safety is easier to encode as reachability exclusion; liveness requires separate machinery

4. The reachability question

Core question

Starting from the initial states S_0, can any execution ever reach a state $s \in U$ (the unsafe set)?

- If **yes**: there is a concrete execution path to a bug — this is a candidate finding
- If **no**: the bug is unreachable — the candidate is a false alarm
- The entire A3 static stage is machinery for answering this question efficiently
- "Efficiently" means: without literally running every possible input

Set notation you will see

R = reachable states, U = unsafe states, goal = prove $R \cap U = \varnothing$

5. True positives and false positives

- When a detector flags a location as "possible bug", it could be:
  - **True positive (TP)**: the bug is real and reachable — an actual threat
  - **False positive (FP)**: the location looks suspicious but the bug cannot happen in practice
  - **False negative (FN)**: a real bug that was missed entirely
- **Alert fatigue**: if a tool produces many FPs, developers stop trusting it and ignore all alerts
- The entire design philosophy of A3 is: **maximise FP elimination before any human sees the result**
- FPs are eliminated by *proving* the unsafe condition is unreachable
- Survivors after elimination are the residue sent to agentic triage

6. Why testing cannot prove absence of bugs

- A function `f(x: int)` has 2^{64} possible inputs on a 64-bit machine
- Testing 1 million inputs per second for a year checks $\approx 3 \times 10^{13}$ inputs — less than 0.00000002% of the space
- A bug on input `x = 1234567890123` would never be found
- **Symbolic execution** sidesteps this by reasoning about *classes* of inputs simultaneously
- **Formal proof** goes further: provides a mathematical guarantee over all inputs
- Real story: the Therac-25 radiation machine killed patients; all manual testing passed; formal analysis would have found the race condition

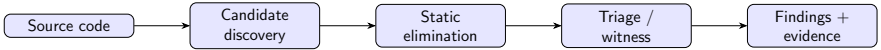
7. Soundness and completeness

- **Sound**: if the analyser says "no bug", there is genuinely no bug (no false negatives)
  - A sound analyser is conservative: it may over-report (some FPs)
- **Complete**: if there is a bug, the analyser will find it (no false negatives either)
  - A complete + sound analyser is perfect — but undecidability limits this for general programs
- **Practically**: most tools are sound-ish (aim for low FN) but not fully sound; A3 is "engineering-grade" — conservative choices, explicit assumptions
- Key phrase to say: "we aim for high-confidence elimination, not omniscient proof"

8. The undecidability wall

- **Rice's theorem**: any non-trivial semantic property of programs is undecidable
- Translation: no algorithm can always correctly answer "does this program reach this state?" for all programs
- This is why verification tools make **approximations**:
  - Over-approximation: consider more states than truly reachable $\Rightarrow$ may add FPs
  - Under-approximation: consider fewer states $\Rightarrow$ may add FNs
  - Time/resource bounds: stop early $\Rightarrow$ return UNKNOWN
- A3 is explicit about which approximations it makes and keeps UNKNOWN outcomes conservatively

9. What a verification pipeline looks like



- **Discovery**: find every location where a bug *could* occur (high recall, many FPs)
- **Static elimination**: prove as many candidates impossible as possible
- **Triage**: classify the survivors with richer context
- **Evidence**: attach proof artifacts or witness traces to each decision

10. Paths and path conditions

- A **path** through a program is a sequence of instructions that could execute together
- Example: in `if x > 0:  return x/y`, there are two paths:
  - Path 1 (taken branch): execute `return x/y` with constraint $x > 0$
  - Path 2 (not-taken branch): skip, no constraint
- A **path condition** Φ is the conjunction of all constraints that must hold for that path to execute
- $\Phi = (x > 0)$ on path 1 — only inputs satisfying this condition reach the division
- Key insight: if the path condition is **unsatisfiable** ($\Phi = \text{false}$), the path is impossible

11. A tiny worked example

```
def f(x, y):
    if x > 0:
        return x / y   # DIV_ZERO?
    return 0
```

- Bug candidate: `x / y` when $y = 0$
- Path condition to reach that line: $x > 0$
- Unsafe condition: $y = 0$
- Combined: $\Phi \wedge U \equiv (x > 0) \wedge (y = 0)$
- Is this satisfiable? **Yes**: take $x = 1, y = 0$
- Conclusion: DIV_ZERO is **reachable** — real bug

12. A guarded version of the example

```
def f(x, y):
    if x > 0 and y != 0:
        return x / y
    return 0
```

- Path condition to reach division: $x > 0 \wedge y \neq 0$
- Unsafe condition: $y = 0$
- Combined: $(x > 0) \wedge (y \neq 0) \wedge (y = 0)$
- Is this satisfiable? **No**: $(y \neq 0) \wedge (y = 0)$ is false
- Conclusion: DIV_ZERO is **unreachable** — false positive, eliminate it
- This is exactly what A3's static stage does at scale

13. The unsafe predicate family

- For each bug type t, there is a predicate $U_t(\sigma)$ that is true exactly when state σ represents a buggy condition
- Examples:
  - $U_{\text{DIV}}(\sigma) \equiv (\text{denominator}(\sigma) = 0)$
  - $U_{\text{OOB}}(\sigma) \equiv (\text{index}(\sigma) \geq \text{len}(\sigma))$
  - $U_{\text{NULL}}(\sigma) \equiv (\text{object}(\sigma) = \texttt{None})$
  - $U_{\text{SQL}}(\sigma) \equiv (\text{taint}(\text{query}) = \texttt{user_controlled})$
- A3 maintains 67+ such predicates covering security and reliability bug classes
- Each eliminates independently: prove $R \cap U_t = \varnothing$ to kill that candidate

14. What "symbolic" means

- **Concrete execution**: run the program with specific input values (e.g. $x = 3, y = 7$)
- **Symbolic execution**: run with *unknown* inputs represented as symbols (e.g. $x = \alpha, y = \beta$)
- As the symbolic program executes, expressions build up: `z = x + 2` becomes $z = \alpha + 2$
- At a branch `if x > 0`: fork into two paths, one with constraint $\alpha > 0$, one with $\alpha \leq 0$
- Result: a tree of paths, each with a path condition, covering all inputs simultaneously
- A satisfiability solver then checks which paths are reachable

15. Over-approximation vs under-approximation (revisited)

- When the analysis is too complex to be exact, we **approximate**
- **Over-approximation** (conservative): pretend the program can reach *more* states than it actually can
  - Risk: extra phantom states may make an unreachable bug look reachable (FP)
  - Benefit: never misses a real bug (sound)
- **Under-approximation** (optimistic): pretend the program can only reach *fewer* states
  - Risk: misses real bugs (FN)
  - Benefit: everything flagged is definitely real (precise)
- A3 over-approximates unknown calls conservatively; concolic under-approximates to confirm witnesses

16. Counterexamples and witnesses

- A **counterexample** (to safety) is a concrete execution trace demonstrating a bug
  - "Call `f(1, 0)`: reaches line 3, divides by zero"
- A **witness** to safety is a proof that no such trace exists
  - "Path condition $\wedge$ unsafe predicate is UNSAT"
- These are the two kinds of evidence A3 attaches to findings in SARIF
- RISE audiences care about: which kind of evidence is this finding backed by?

17. The state space

- A program's **state space** is the set of all possible (program counter, variable values, heap, ...) combinations
- State spaces are astronomically large (or infinite)
- Verification's fundamental technique: reason about **sets of states** abstractly instead of individual states
- Many methods characterise the reachable set R implicitly via an inductive property
- **Key challenge**: the reachable set may be a complex shape (not a simple box/interval)

18. Inductive vs non-inductive arguments

- A **non-inductive safety argument**: enumerate all paths and check each one
  - Only works for programs with a finite, small number of paths
  - Fails on loops (infinite paths)
- An **inductive safety argument**: find a property P such that:
  1. P holds at the start (initialisation)
  2. If P holds now and the program takes one step, P still holds (consecution)
  3. P implies safety (the unsafe states are outside P)
- If you can find such a P, you have proved safety for *all* iterations of loops, not just finite unrollings
- This triple is the foundation of every method in the A3 portfolio

19. The word "consecution"

Definition

Consecution (also called the *inductive step*) is the property:

$$\text{if } P(s) \text{ holds and } s \to s', \text{ then } P(s') \text{ holds}$$

- The name comes from "consequence" + "succession" — the property is *preserved through successive steps*
- You will use this word a lot in the A3 talk: "the consecution obligation"
- Intuition: once you are inside the safe zone, one program step cannot throw you out
- If consecution holds for all transitions, the program can never escape to an unsafe state

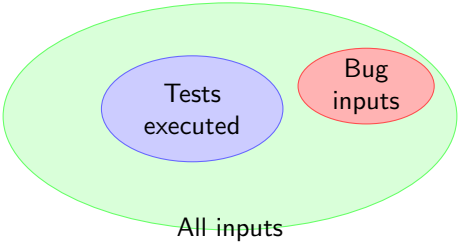
20. Why consecution is hard

- For a simple assignment `x = x + 1`, consecution check: does $P(x) \Rightarrow P(x+1)$?
- For a while loop with complex body, the transition $s \to s'$ involves many instructions
- For function calls, the transition may involve unknown code (library calls)
- For Python: exception edges, heap mutations, dynamic dispatch all make "one step" complex
- **Main technical work** in verification is defining what "one step" means precisely enough to check consecution

21. Safety as set separation

- Rephrase: find a set S (the "safe zone") such that:
  1. Initial states are inside S: $S_0 \subseteq S$
  2. S is closed under transitions: $s \in S, s \to s' \Rightarrow s' \in S$
  3. S and the unsafe set U do not overlap: $S \cap U = \varnothing$
- The set S is called an **inductive invariant**
- Finding such a set *proves* the program is safe forever
- Many names for this: Lyapunov function analog, barrier certificate, loop invariant, separating predicate

22. The gap between testing and verification



- Tests cover a tiny region; bugs may live entirely outside it
- Verification reasoning covers the whole space (or provably excludes the red zone)

23. Bug classification in A3

- A3 groups bugs into two families:
- **Security bugs** (taint-based): unsafe data flow from untrusted input to dangerous operation
  - SQL injection, command injection, path traversal, XSS, ...
- **Reliability bugs** (error-based): program state violates safety invariant at execution time
  - DIV_ZERO, INDEX_OUT_OF_BOUNDS, NULL_DEREF, UNCAUGHT_EXCEPTION, ...
- Each family uses a different unsafe predicate shape: taint predicates vs numeric/type predicates
- Both are handled by the same barrier/SMT machinery; only the predicate changes

24. What SARIF is

- **SARIF** = Static Analysis Results Interchange Format (Microsoft/OASIS standard)
- A JSON format for recording analysis findings with structured evidence
- Key fields: rule ID, file + line, severity, confidence, message, evidence chain
- A3 uses SARIF to pass findings from the static stage to triage and then to CI
- Benefit: findings are reproducible, diffable, and queryable — not just a text report
- When you say "SARIF proof artifact" in the talk, it means: the JSON record includes the solver outcome or certificate that justified elimination

25. Chapter 1 summary: vocabulary you now know

Core concepts:

- Bug, safety property, reachability
- True positive / false positive
- Path condition Φ
- Unsafe predicate U_t
- State space, symbolic execution

Key properties:

- Soundness, completeness
- Over/under approximation
- Inductive invariant
- Consecution (inductive step)
- Counterexample, witness

One-sentence summary

Verification proves or disproves that bad states are reachable; A3 does this at scale by building path conditions and checking them with solvers.

Chapter 2: Logic, SAT, and SMT Solvers

Chapter goal

Understand what a solver is, what UNSAT means, and why Z3 is the engine behind A3's elimination step.

26. Propositional logic in 3 minutes

- **Atoms**: Boolean variables: p, q, r (true or false)
- **Connectives**: $\neg p$ (not), $p \wedge q$ (and), $p \vee q$ (or), $p \Rightarrow q$ (implies)
- **Formula**: a combination of atoms and connectives
- **Assignment**: mapping each atom to true/false
- **Satisfying assignment**: one that makes the formula true
- Example: $(p \vee q) \wedge \neg p$ is satisfied by $p = \text{false}, q = \text{true}$

27. SAT: the satisfiability problem

SAT

Given a propositional formula φ, is there an assignment that makes φ true?

- **SAT** (Satisfiable): at least one satisfying assignment exists
- **UNSAT** (Unsatisfiable): no assignment makes it true — the formula is logically impossible
- Example UNSAT formula: $p \wedge \neg p$ — requires p true and false simultaneously
- SAT is NP-complete, but modern SAT solvers handle millions of variables in practice
- SAT solvers appear everywhere in verification (IC3/PDR especially)

28. From SAT to SMT

- SAT works over Boolean variables only
- Programs have **integers, reals, strings, arrays** — we need richer logic
- **SMT** (Satisfiability Modulo Theories): SAT extended with theories for:
  - **Linear arithmetic**: $2x + 3y \leq 7$, integers or reals
  - **Nonlinear arithmetic**: $x^2 + y^2 \leq 1$
  - **Arrays**: $a[i] = v$
  - **Bit-vectors**: $x \,\&\, 0xFF = 0$
  - **Strings**: regular membership, concatenation
- **SMT solver**: decides satisfiability over these richer formulas

29. Z3: the solver A3 uses

- **Z3** is an SMT solver developed by Microsoft Research (de Moura & Bjørner, 2008)
- Nikolaj Bjørner is literally named as a target user in the project's origin story
- Z3 handles integer arithmetic, real arithmetic, arrays, bit-vectors, strings, and more
- In A3, Z3 is used to:
  1. Check if $\Phi \wedge U_t$ is satisfiable (is the bug reachable?)
  2. Solve for barrier certificate coefficients c_α
  3. Check CHC constraints (consecution in interprocedural form)
- Z3 returns: `sat` + model, `unsat` + proof, or `unknown`

30. What UNSAT actually means for us

When Z3 says UNSAT

The formula $\Phi(\sigma) \wedge U_t(\sigma)$ is **unsatisfiable**: no state can simultaneously satisfy the path condition *and* be unsafe.

- This is a **proof of unreachability**: the bug cannot happen on this path
- The UNSAT proof is the "elimination evidence" stored in the SARIF artifact
- When you say "UNSAT kills the candidate", you mean: the solver certified impossibility
- UNSAT is trustworthy (up to solver correctness) — no need for human review

31. What SAT means for us

When Z3 says SAT + model

The formula $\Phi(\sigma) \wedge U_t(\sigma)$ is satisfiable. The model gives specific values (e.g. $x = 1, y = 0$) that make both conditions true simultaneously.

- SAT: the bug **might** be reachable (possibly after more checking)
- The model becomes a **witness seed** — it specifies input values to try concretely
- SAT does not always mean definite bug: the symbolic model may not correspond to a real execution (spurious SAT can arise from over-approximation)
- Action: retain the candidate; pass model to concolic stage to confirm

32. What UNKNOWN means for us

- Z3 may return `unknown` when it cannot decide within resource limits
- Causes: timeout, nonlinear arithmetic beyond solver's reasoning power, quantifier alternation
- **Conservative policy**: treat UNKNOWN as SAT — retain the candidate, escalate to the next portfolio method
- This is the **soundness-preserving choice**: if we pruned on UNKNOWN, we might silently miss real bugs
- UNKNOWN is not a failure; it is a routing signal to a stronger/different method

33. First-order logic and quantifiers

- Beyond propositional logic, we also use **first-order logic (FOL)**:
  - $\forall x : P(x)$ — "for all values of x, property P holds"
  - $\exists x : P(x)$ — "there exists a value of x such that P holds"
- The barrier certificate obligations use $\forall$: "for all initial states", "for all transitions"
- Z3 can handle many $\forall/\exists$ formulas over arithmetic via quantifier instantiation
- Full first-order arithmetic is undecidable, which is why Z3 sometimes returns UNKNOWN

34. Clauses, conjunctive normal form (CNF)

- A **clause** is a disjunction of literals: $(x > 0) \vee (y \neq 0) \vee \neg p$
- **CNF** = conjunction of clauses: $C_1 \wedge C_2 \wedge \ldots \wedge C_n$
- SAT solvers work natively in CNF
- In IC3/PDR, the inductive invariant is maintained as a conjunction of clauses — this is why IC3 is sometimes described as a "clause-learning" method
- When you hear "blocking clause" or "proof obligation as clause", it refers to this CNF structure

35. Implications and implies-false

- $A \Rightarrow B$ is equivalent to $\neg A \vee B$
- To prove $A \Rightarrow$ false (i.e., A is impossible), check that A is UNSAT
- This is how the solver checks safety: is $(\Phi \wedge U)$ satisfiable? Equivalent to: does $\Phi \Rightarrow \neg U$ fail?
- All three barrier obligations are formulated as implications checked by the solver:
  - Init: $s \in S_0 \Rightarrow B(s) \geq \epsilon$
  - Consec: $B(s) \geq 0 \wedge T(s, s') \Rightarrow B(s') \geq 0$
  - Sep: $s \in U \Rightarrow B(s) \leq -\epsilon$

36. Quantifier-free linear arithmetic (QF_LIA / QF_LRA)

- Most path conditions in A3 involve linear constraints (no multiplication of variables)
- **Quantifier-free linear integer arithmetic** (QF_LIA): formulas like $2x + y \leq 7$
- These are **decidable**: Z3 will always return SAT or UNSAT (never UNKNOWN) given enough time
- Most guard-based elimination in A3 falls in this decidable fragment
- UNKNOWN only arises from: nonlinear constraints, quantifiers over infinite domains, or timeout

37. Nonlinear arithmetic and why it is hard

- **Nonlinear arithmetic**: formulas involving products of variables, e.g. $x \cdot y = 0$
- Polynomial arithmetic over reals (quantifier-free) is decidable (cylindrical algebraic decomposition) but expensive (DOUBLY EXPONENTIAL in the number of variables)
- Polynomial arithmetic over integers is undecidable (related to Hilbert's 10th problem)
- This is why barrier certificates (which use polynomials) are expensive: they operate in nonlinear arithmetic land
- DSOS/SDSOS exist precisely to tame this cost with cheaper approximations

38. What a model looks like

- When Z3 says SAT, it also returns a **model**: a concrete assignment of values
- Example: for $(x > 0) \wedge (y = 0) \wedge (x < 10)$, model might be $\{x \mapsto 1, y \mapsto 0\}$
- This immediately gives us **test inputs** that would trigger the bug
- A3's concolic stage takes this model and executes the program concretely with those values
- If the concrete execution actually hits the unsafe state, the bug is confirmed real

39. Proof certificates from SMT solvers

- When Z3 returns UNSAT, it can optionally produce a **proof certificate**: a checkable derivation showing exactly which axioms and steps led to contradiction
- This is the mathematical guarantee that the UNSAT result is correct
- In practice, A3 uses the UNSAT result as evidence; full proof certificate generation is expensive and used selectively
- **UNSAT core**: a minimal subset of the input formula that is already UNSAT — useful for understanding *which* constraints caused the contradiction (e.g. which guard killed the bug)

40. Craig interpolants: preview

- Given formula $A \wedge B$ is UNSAT, a **Craig interpolant** I satisfies:
  - $A \Rightarrow I$ (the interpolant follows from A)
  - $I \wedge B$ is UNSAT (the interpolant contradicts B)
  - I only uses symbols that appear in *both* A and B
- Intuition: I is the "shared reason" why A and B conflict
- In verification: A = path from start, B = unsafe condition; I = new safe-zone boundary predicate
- You will see this in slides 13f: interpolants as the tightest possible new template constraint

41. Taint analysis in brief

- **Taint analysis** tracks whether data is "tainted" (comes from an untrusted source)
- **Source**: where tainted data enters (e.g. HTTP request parameter, file input)
- **Sink**: where tainted data would be dangerous (e.g. SQL query, shell command)
- **Sanitizer**: a function that cleans tainted data (e.g. `escape()`, `parameterize()`)
- Taint unsafe predicate: $U_{\text{SQL}}(\sigma) \equiv (\text{taint}(\text{arg}(\text{query})) = \text{unsanitized})$
- A3 maintains a taint lattice through symbolic execution; the lattice has levels source/unsanitized/sanitized/safe

42. The solver loop in A3: recap

Step-by-step solver interaction

1. Build symbolic state σ at candidate bug site
2. Extract path condition $\Phi(\sigma)$ from symbolic execution
3. Form query: $\Phi(\sigma) \wedge U_t(\sigma)$
4. Send to Z3
5. **UNSAT**: add to eliminated set with evidence
6. **SAT + model**: retain; seed concolic with model values
7. **UNKNOWN**: retain; route to next portfolio stage

43. Why Z3 and not other solvers?

- Z3 is open-source, actively maintained, and has excellent Python bindings (`z3-solver` on PyPI)
- It covers the widest theory combination relevant to Python programs: integers, reals, arrays, bit-vectors, strings
- It supports incremental solving (add/remove constraints without restarting) — important for efficiency in loops
- Alternative solvers: CVC5, MathSAT, Yices2 — each has strengths; A3 currently uses Z3 as primary
- Nikolaj Bjørner (Z3 lead) is explicitly in the project origin story: "Z3-builder level operational usefulness"

44. Incremental solving and push/pop

- **Incremental solving**: add constraints progressively; Z3 remembers learned facts
- **push**: save current solver state (like a checkpoint)
- **pop**: restore to last checkpoint
- This is efficient for checking many candidates that share a common prefix
- Example: a function has 5 bug candidates, all under the same entry path condition. Push the shared Φ, check each U_t without restarting

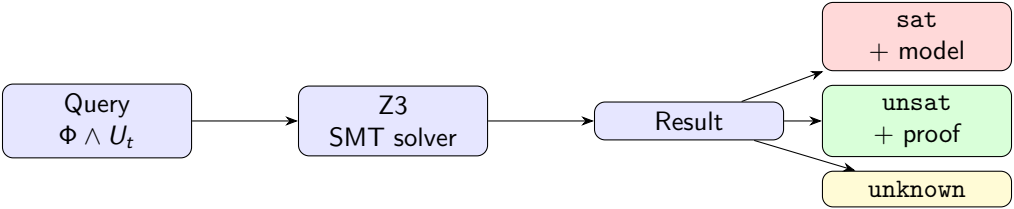
45. Theories combination: what it means in practice

- Python programs mix integers, booleans, strings, objects, and exceptions in one expression
- A path condition might look like: $(x > 0) \wedge (\text{isinstance}(x, \texttt{int})) \wedge (s = \texttt{"hello"} + t)$
- SMT solvers handle this via **theory combination** (Nelson–Oppen method)
- Different theory solvers cooperate: arithmetic solver handles numeric parts; string solver handles string parts
- Combined solver is still sound; performance depends on how much cross-theory interaction there is
- A3's SymbolicValue carries a type tag (`INT`, `REAL`, `NONE`, object-id) to route constraints to the right theory

46. The notion of a formula being "valid"

- A formula φ is **valid** if it is true under *every* assignment
- Equivalent to: $\neg\varphi$ is UNSAT
- This is how we prove the barrier certificate obligations: to prove $\forall s : A(s) \Rightarrow B(s)$, negate it to $\exists s : A(s) \wedge \neg B(s)$ and ask Z3 if that is UNSAT
- If UNSAT: the implication is valid (the obligation holds)
- If SAT: the model gives a counterexample to the obligation

47. SMT in one picture



48. Practical SMT pitfalls

- **Quantifier explosion**: $\forall x$ over an infinite domain requires instantiation heuristics; can cause UNKNOWN
- **Nonlinear products**: $x \cdot y$ triggers expensive procedures; prefer linear approximations where possible
- **Timeout**: Z3 has configurable timeouts; hitting the limit returns UNKNOWN
- **Encoding bugs**: if the formula does not faithfully encode the program semantics, the solver's answer is meaningless
- A3 invests heavily in correct encoding; every bytecode operation has a precise symbolic transfer function

49. Z3 Python API: a quick look

```
from z3 import *
x, y = Ints('x y')
solver = Solver()
solver.add(x > 0)           # path condition
solver.add(y == 0)          # unsafe predicate (DIV_ZERO)
result = solver.check()     # -> sat or unsat
if result == sat:
    print(solver.model())   # e.g. [x = 1, y = 0]
elif result == unsat:
    print("Bug unreachable - FP eliminated")
```

- This is literally what A3 does for a simple guard check
- Real queries are much larger (hundreds of constraints from symbolic execution)

50. Chapter 2 summary: solver vocabulary

Logic terms:

- SAT, UNSAT, UNKNOWN
- SMT, Z3, theory
- Model (satisfying assignment)
- Proof certificate
- Valid formula

Key decisions in A3:

- UNSAT $\Rightarrow$ eliminate
- SAT $\Rightarrow$ retain + witness seed
- UNKNOWN $\Rightarrow$ retain conservatively
- QF_LIA is decidable; nonlinear is hard
- Craig interpolant = tight new predicate from UNSAT proof

Chapter 3: Symbolic Execution

Chapter goal

Learn how a verifier "runs" a program on abstract inputs, builds path conditions, and uses them to check whether a bug is reachable. Understand path explosion and how A3 manages it.

51. Concrete vs symbolic execution: side by side

Concrete run ($x = 3$):

```
z = x + 1     # z = 4
if z > 3:     # true
  w = z * 2 # w = 8
```

Symbolic run ($x = \alpha$):

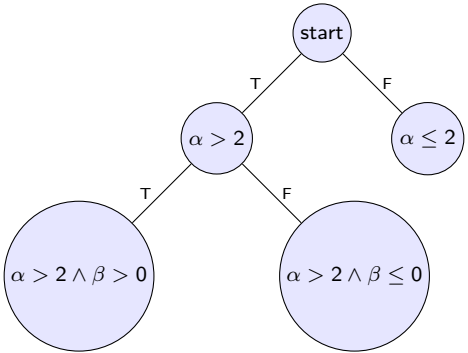
```
z = alpha + 1    # z = alpha+1
if z > 3:        # fork:
  # branch T: alpha+1 > 3 => alpha > 2
  w = (alpha+1)*2
  # branch F: alpha+1 <= 3 => alpha <= 2
  # skip
```

- Symbolic execution covers **all** values of α in one pass
- Each branch gets its own path condition

52. The symbolic state

- At any point in symbolic execution, the **symbolic state** records:
  - **Symbolic store** ρ: each variable mapped to a symbolic expression (not a value) e.g. $\rho = \{x \mapsto \alpha, z \mapsto \alpha + 1, w \mapsto 2(\alpha + 1)\}$
  - **Path condition** Φ: conjunction of branch constraints accumulated so far e.g. $\Phi = (\alpha > 2)$
  - **Program counter** pc: which instruction executes next
- At a potential bug site (e.g. division), extract the symbolic denominator $d = \rho[\text{denom}]$
- Bug-reachability query: is $\Phi \wedge (d = 0)$ satisfiable?

53. Branch forks and the path tree



- Each branch doubles the number of active paths
- A function with n independent binary branches has up to 2^n paths
- **Path explosion**: infeasible to enumerate all paths in a large function

54. Path explosion and how to manage it

- **Loops**: potentially infinite paths (each iteration is a new fork)
- **Mitigations**:
  1. **Loop unrolling bound**: explore at most k loop iterations (under-approximation)
  2. **Merge paths**: join states after branches with a disjunction (over-approximation)
  3. **Widening**: abstract loops into an over-approximate fixed point
  4. **Inductive invariants**: replace loop body with a proof obligation (exact but requires finding invariant)
- A3 uses loop unrolling for SMT queries and inductive barrier certificates for loops with infinite iteration

55. How branches become path condition constraints

```
def f(items, idx):
    if 0 <= idx:
        if idx < len(items):
            return items[idx]
    return None
```

Path to `items[idx]` (line 4):

- From branch 1 (true): $0 \leq \alpha$
- From branch 2 (true): $\alpha <$ len(items)
- Symbolic index: α
- OOB check: $\alpha \geq$ len(items)
- Query: $(0 \leq \alpha) \wedge (\alpha < L) \wedge (\alpha \geq L) =$ UNSAT
- Conclusion: access is safe, FP eliminated

56. Symbolic expressions and their representations

- A symbolic expression is a **formula tree**: leaves are symbolic variables or constants, nodes are operations
- Example: `(x + 2) * y - 1` becomes $(\alpha + 2) \cdot \beta - 1$ as a tree
- A3's `SymbolicValue` class wraps a Z3 expression
- Operations on `SymbolicValue` build larger Z3 expressions: `sv + 1` $\Rightarrow$ `z3_expr + z3.IntVal(1)`
- The path condition Φ is also a Z3 expression (boolean) grown with `solver.add()` calls

57. Exception edges as paths

- Python is full of implicit exceptions: `TypeError`, `AttributeError`, `KeyError`, ...
- An exception is another kind of branch: the "exceptional path" vs the "normal path"
- Example: `x.attr` — two paths:
  - Normal path: $x \neq$ `None`, execution continues
  - Exception path: $x =$ `None`, `AttributeError` raised, unwinds to nearest handler
- A3's symbolic execution tracks both paths; `UNCAUGHT_EXCEPTION` is unsafe when the exception path leads out of the program with no handler

58. Symbolic heap and object identity

- Python objects live on the heap; variables hold **references** (object IDs)
- A3 uses symbolic object IDs: variable `x` holds ID ι_x (an integer in the symbolic store)
- Alias analysis: could two variables hold the same object? $\iota_x = \iota_y$? (can be modelled symbolically)
- Heap mutation (e.g. `x.field = v`): updates symbolic map from object-ID to field values
- This is expensive; A3 uses conservative approximations for complex heap structures

59. Function calls in symbolic execution

- **Inlined functions**: substitute body into caller, continue symbolic execution
- **Summarised functions**: replace body with a pre-computed summary (pre/postcondition pair)
- **Unknown functions** (library calls, C extensions): use a conservative abstraction such as "return any value" or a hand-written stub
- A3 inlines up to a depth bound; beyond that, uses stubs or returns UNKNOWN
- The choice between inline/summarise affects both precision and cost

60. What A3's symbolic execution engine produces

- For each function, symbolic execution produces a set of **(path condition, symbolic state)** pairs
- One pair per feasible path through the function body up to the bounded depth
- For each bug candidate t at location ℓ, the engine extracts:
  - Which paths reach ℓ?
  - On each such path, what is the path condition?
  - What is the symbolic value of the relevant expression (denominator, index, etc.)?
- This feeds directly into the Z3 query from Chapter 2

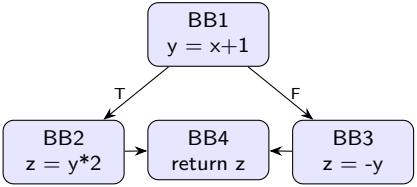
Chapter 4: Control Flow Graphs and Program State

Chapter goal

Understand CFGs, basic blocks, the state tuple, and transition systems — the formal foundation that A3's barrier certificate machinery is built on.

61. Control flow graph (CFG)

```
def g(x):
    y = x + 1    # BB1
    if y > 0:    # BB1 end
        z = y * 2    # BB2
    else:
        z = -y       # BB3
    return z     # BB4
```



62. Basic blocks

- A **basic block** (BB) is a straight-line sequence of instructions with:
  - Exactly one entry point (first instruction)
  - Exactly one exit point (last instruction: conditional branch, unconditional jump, or return)
- Within a basic block: no branches, no loop-backs — execution is sequential
- CFG edges capture **control flow** between blocks
- Most analysis algorithms work at the granularity of basic blocks

63. The state tuple σ

A3's program state tuple

$$\sigma = (\text{pc},\ \rho,\ h,\ \tau,\ \kappa,\ e,\ \Phi)$$

- **pc**: program counter — which instruction is next
- ρ: local variable store (symbolic values)
- h: heap (object ID $\rightarrow$ field values)
- τ: type environment (object ID $\rightarrow$ type)
- κ: taint tracking lattice values
- e: exception state (None or pending exception object)
- Φ: accumulated path condition

64. Transition relation

- The **transition relation** $T(\sigma, \sigma')$ is true when the program can go from state σ to state σ' in one step
- Each bytecode instruction defines how T updates:
  - `LOAD_FAST x`: $\sigma' = \sigma[\text{stack} \mathrel{+}= \rho[x]]$
  - `BINARY_ADD`: $\sigma' = \sigma[\text{stack} \mathrel{+}= v_1 + v_2]$
  - `POP_JUMP_IF_TRUE addr`: fork two states with added constraints
- The symbolic execution engine implements exactly this: apply instruction transfer functions
- Consecution over T is the key obligation: does the safe zone survive one step?

65. Initial states S_0

- S_0 is the set of states at the function entry point with all inputs unconstrained (fully symbolic)
- Formally: $S_0 = \{(\text{pc} = \text{entry}, \rho = \text{fresh symbols}, h = \text{empty}, \ldots, \Phi = \text{true})\}$
- "$\Phi = \text{true}$" means: no constraints on inputs yet — anything is possible
- As execution proceeds, constraints accumulate in Φ
- Barrier certificates must hold for *all* states in S_0 — the init obligation

66. Reachable states R and the fixed point

- The **reachable set** R is defined inductively:

$$R_0 = S_0, \quad R_{i+1} = R_i \cup \{\sigma' : \exists \sigma \in R_i,\ T(\sigma, \sigma')\}$$

$$R = \bigcup_{i=0}^{\infty} R_i$$

- R is the fixed point of the least-fixpoint equation: $R = S_0 \cup \mathrm{post}(T, R)$
- For infinite-state programs (loops), R may be infinite even if the program terminates
- A **sound over-approximation** $\hat{R} \supseteq R$ is sufficient to prove safety

67. Why loops make reachability hard

- At a loop head, states accumulate from: entry (first pass) and back-edge (subsequent passes)
- After k iterations, the reachable set at the loop head is the union of $k + 1$ sets
- For $k \to \infty$: the loop head reachable set requires an infinite fixed point
- Techniques to compute this fixed point:
  1. **Widening**: aggressive over-approximation, terminates quickly but imprecise
  2. **Inductive invariant**: find a predicate I that is preserved by the loop body
  3. **Template + solver**: parameterise I by unknown coefficients; solve for them
- A3 uses template-based barrier certificates to handle loops

68. Loop invariants in practice

- A **loop invariant** is a predicate I such that:
  1. I holds before the loop starts (init)
  2. If I holds at the top of the loop and the body executes, I holds again at the top (consecution over the loop body)
- I also implies the property you care about at all loop exits
- Example: for a counting loop `for i in range(n): arr[i] = ...`, invariant $0 \leq i \leq n$ ensures OOB-safety
- Finding I automatically is the hard problem (requires synthesis)

69. A worked loop invariant

```
i = 0
while i < n:
    arr[i] = i * 2
    i += 1
# want: no OOB
```

Proposed invariant: $I \equiv (0 \leq i \leq n)$

- Init: $i = 0$ so $0 \leq 0 \leq n$ holds (if $n \geq 0$)
- Consecution: inside loop, $i < n$ (guard), so after $i{+}{+}$, still $i \leq n$. ✓
- Safety: $0 \leq i < n$ when accessing `arr[i]`, so no OOB. ✓

This is an inductive invariant that proves safety.

70. Interprocedural analysis

- **Intraprocedural**: analyse one function in isolation (good for simple properties)
- **Interprocedural**: analyse across function call boundaries
- Why it matters: a bug often spans multiple functions: user input arrives in `handle_request()`, passes through `process()`, reaches `execute_query()` — SQL injection needs full interprocedural tracking
- **Approach 1**: inline callees (expensive; blows up for recursive/large programs)
- **Approach 2**: compute **summaries** (pre/postconditions) per function; compose
- A3 produces per-function summaries; interprocedural taint flows use summary chaining

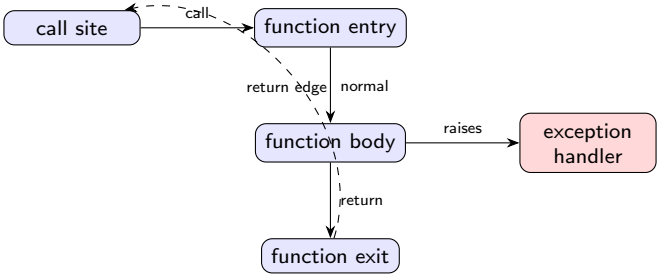
71. Function summaries

- A **summary** for function f is a relation $S_f(\mathbf{in}, \mathbf{out})$ capturing what f can do without specifying the body
- Example: `len(xs)` summary: $\mathbf{out} = \text{length}(\mathbf{in})$, always ≥ 0
- **Modular verification**: verify each function independently using summaries of callees
- **Contract-based**: function contracts are a special case of summaries where developer writes them
- A3 can use hand-specified stubs (for library calls) and auto-inferred summaries (for project code)

72. What the A3 transition system looks like precisely

- **State**: $\sigma = (\mathrm{pc}, \rho, h, \tau, \kappa, e, \Phi)$ as defined earlier
- **Initial states**: $S_0 = \{\sigma_0\}$ where σ_0 has fresh symbolic inputs and empty constraints
- **Transition**: bytecode-level small-step semantics, one instruction at a time
- **Unsafe predicate**: $U_t(\sigma)$ — depends on bug type t
- **Goal**: prove $\forall \sigma \in R : \neg U_t(\sigma)$, equivalently find inductive invariant I that separates R from U_t
- The barrier certificate is exactly this invariant, but with a polynomial encoding that Z3 can reason about

73. CFG edges: normal, exception, call, return



- Exception edges are often the path through which bugs propagate
- A3 tracks exception paths symbolically, not just normal control flow

74. Program slicing for focused analysis

- A **program slice** w.r.t. a bug candidate at location ℓ keeps only the instructions that could influence the variables relevant to the bug
- Example: for DIV_ZERO at a/b, keep only code that assigns to or affects b's value
- **Backward slice**: walk backward through data-flow edges from ℓ
- Slicing dramatically reduces the size of the symbolic execution problem
- A3 performs lightweight slicing before generating the Z3 query

75. Chapter 3+4 summary: execution and state model

Symbolic execution:

- Symbolic store + path condition
- Branch fork creates path tree
- Path explosion managed by bounds/invariants
- Exception edges are paths too

Program model:

- CFG: basic blocks + edges
- State tuple $(\mathrm{pc}, \rho, h, \tau, \kappa, e, \Phi)$
- Transition relation T
- Reachable set R = fixed point
- Loop $\Rightarrow$ inductive invariant needed

Chapter 5: Inductive Invariants and Consecution

Chapter goal

Master the concept of consecution deeply. Understand loop invariants, the induction pattern, and k-induction — the foundation of every formal proof of safety.

76. The three-property safety proof pattern

Inductive invariant safety proof

To prove $S_0 \cap T^* \to U = \varnothing$, find a set I with:

$$\underbrace{S_0 \subseteq I}_{\text{(I) Initiation}} \qquad \underbrace{I \times I \supseteq T \cap (I \times All)}_{\text{(C) Consecution}} \qquad \underbrace{I \cap U = \varnothing}_{\text{(S) Separation}}$$

- **Initiation**: every initial state is inside I
- **Consecution**: the image of I under T stays inside I
- **Separation**: I never overlaps the unsafe region

These three properties prove that no execution can ever reach U.

77. Why all three obligations are necessary

- **Without Initiation**: I might be safe but the program starts outside it — useless
- **Without Consecution**: I might hold at start but the program escapes I on step 2
- **Without Separation**: I overlaps U — we cannot conclude the bug is unreachable
- Think of I as a **fence**: init says "program starts inside the fence"; consecution says "the fence is sturdy enough to keep the program in"; separation says "the fence excludes the danger zone"
- All three fences must hold simultaneously

78. A simple invariant example: counting

- Program: increment counter `n` from 0 to 100; check at each step that $n \leq 100$
- Proposed invariant: $I \equiv (0 \leq n \leq 100)$
- Initiation: $n = 0 \Rightarrow 0 \leq 0 \leq 100$. ✓
- Transition: $n \to n + 1$, guard is $n < 100$
- Consecution: $(0 \leq n \leq 100) \wedge (n < 100) \Rightarrow (0 \leq n + 1 \leq 100)$. ✓
- Separation: I clearly avoids $U \equiv (n > 100)$. ✓
- Proof complete — counter never exceeds 100

79. Consecution as a query

- The consecution check is: is this formula UNSAT?

$$I(\sigma) \wedge T(\sigma, \sigma') \wedge \neg I(\sigma')$$

- This says: "I holds now, the program takes one step, but I fails after the step" — if this has no solution, consecution holds
- Send to Z3: `solver.check(I(s) ∧ T(s,s') ∧ not I(s'))`
- If UNSAT: consecution proved
- If SAT: the model gives a specific pre-state and post-state that violate consecution — a counterexample to help refine I

80. The induction metaphor

- Mathematical induction: to prove $P(n)$ for all $n \geq 0$:
  1. Prove $P(0)$ (base case)
  2. Prove $P(n) \Rightarrow P(n+1)$ (inductive step)
- Program inductive invariant is exactly the same:
  1. Base case = Initiation: I holds at step 0
  2. Inductive step = Consecution: if I holds at step n, it holds at step $n+1$
- The "step" here is one execution step of the program (one instruction or one loop iteration)
- This metaphor helps explain why a proof works for *all* iterations of a loop, not just a bounded number

81. When consecution is easy vs hard

- **Easy**: assignments with linear expressions; direct SMT query on linear arithmetic
- **Hard**: loops with complex nonlinear update (e.g. $x \leftarrow x^2 + y$)
- **Hard**: heap mutations that change aliasing structure
- **Hard**: dynamic dispatch (which method actually runs?) — over-approximated by considering all possible callees
- **Hard**: unknown side effects from unanalysed library calls
- A3 uses a portfolio: simple linear invariants first (fast), barrier certificates for nonlinear, UNKNOWN routing for the rest

82. The simplest inductive invariant: the guard itself

- Often the simplest invariant is the **guard condition** that protects the bug
- Example: `if y != 0: result = x / y` — guard is $y \neq 0$
- Proposed I: "we can only reach the division when $y \neq 0$" encodes the guard in the invariant
- This is essentially what the path-condition check does: verify that $\Phi \Rightarrow \neg U_t$
- For more complex programs, the invariant must be synthesised because the guard is not immediately at the bug site

83. k-induction: relaxing the base case

- **Simple induction** requires: $I(\sigma) \Rightarrow I(\sigma')$
- **k-induction**: prove that if I holds for k consecutive states, it holds for the $(k+1)$-th:

$$I(\sigma_0) \wedge I(\sigma_1) \wedge \ldots \wedge I(\sigma_{k-1}) \wedge T^k(\sigma_0, \sigma_k) \Rightarrow I(\sigma_k)$$

- Why? Some properties do not satisfy simple induction but do satisfy k-induction for a specific k
- **Example**: a property that alternates between two "safe" sub-states requires $k = 2$ to prove
- A3 can escalate from $k = 1$ to higher k when simple induction fails

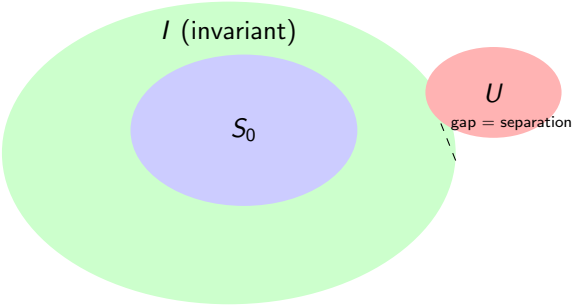
84. k-induction worked example

- Program: two state machine variables a, b alternating: each step $(a, b) \to (b, a + b)$
- Safety property: $a > 0 \wedge b > 0$, starting from $a = 1, b = 1$
- Simple induction check: $(a > 0 \wedge b > 0) \Rightarrow (b > 0 \wedge (a + b) > 0)$ — holds! ($\checkmark$)
- So simple induction works here; some cases genuinely need $k > 1$
- Key point: **k-induction is strictly more powerful** — it can prove properties that simple induction cannot, while being a sound technique

85. Strengthening invariants

- Sometimes I fails consecution: the post-state escapes I for some transition
- The model (counterexample to consecution) gives a clue about what is missing
- **Invariant strengthening**: add an additional conjunct to I to block that counterexample
- Example: $I_1 = (n \geq 0)$ fails; SAT model shows $n = -1$ is reachable via exception path
- Strengthen to $I_2 = (n \geq 0) \wedge (\text{exception} = \text{None})$
- Repeat until consecution holds or we give up (UNKNOWN)
- This iterative strengthening is the basis of CEGAR (Chapter 7)

86. The ghost of the inductive invariant: what it looks like geometrically



- $S_0 \subseteq I$ (initiation): small blue inside big green
- $I \cap U = \varnothing$ (separation): green and red do not touch
- The transition keeps all reachable states inside the green ellipse

87. Relationship between invariants and barrier certificates

- An **inductive invariant** is any predicate satisfying the three obligations
- A **barrier certificate** is a specific kind of inductive invariant where:
  - The invariant is represented as $\{s : B(s) \geq 0\}$ for a *function* B
  - B is a polynomial — enabling sum-of-squares (SOS) methods to find and verify it
- The barrier certificate is an *implementation* of the inductive invariant concept
- You can describe it to the RISE audience as: "we find an inductive invariant automatically using polynomial optimisation, which the solver can then certify"

88. When is a simple invariant not enough?

- For linear programs (only additions, subtractions, constants) linear invariants suffice
- For nonlinear programs (multiplications, squares, transcendental functions) linear invariants may be too weak
- Example: safety of a square root function requires knowing $x \geq 0$; a linear invariant over x works
- But: safety of a program with $z = x * y$ may require invariants like $z \leq x^2 + y^2$ (nonlinear)
- This is where SOS polynomial templates are needed — covered in Chapter 6

89. The role of UNKNOWN in invariant finding

- If the solver cannot decide whether a proposed invariant is inductive:
  1. Timeout on the consecution query
  2. Nonlinear terms beyond solver's reasoning power
- Result: UNKNOWN
- Policy: route to next portfolio method (e.g. try a richer polynomial template)
- If all portfolio methods return UNKNOWN: report candidate conservatively (do not call it FP)
- This preserves soundness: we only eliminate when we have a real proof

90. Example: proving DIV_ZERO unreachable with an invariant

- Function: compute `mean(data)` — divides `sum` by `count`
- Candidate: DIV_ZERO at `sum / count`
- Proposed invariant: $I \equiv (\texttt{count} = \text{len}(\texttt{data}))$
- This invariant says: count equals the number of elements processed
- At the division site, we also need: $\text{len}(\texttt{data}) > 0$ (precondition of function)
- With both: $\texttt{count} = \text{len}(\texttt{data}) > 0 \Rightarrow \texttt{count} > 0$ — no DIV_ZERO
- The invariant is proved inductive over the accumulation loop; separation holds given the precondition

91. Contracts and preconditions

- A **precondition** is a constraint on the inputs that the caller must guarantee
- A **postcondition** is a constraint on the output that the callee must guarantee
- Together they form a **contract** (Hoare triple: $\{P\}$ program $\{Q\}$)
- In the `mean` example: precondition len(`data`) > 0 is required
- A3 can use contracts to:
  - Assume preconditions when analysing a callee (reduce over-approximation)
  - Propagate postconditions to callers (compose summaries)

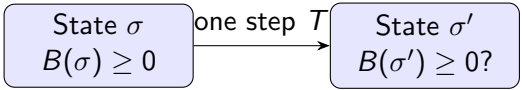
92. The Hoare triple

Hoare triple

$\{P\}\ C\ \{Q\}$: if precondition P holds before executing C, then postcondition Q holds after

- **Partial correctness**: assumes C terminates; only asserts Q if it does
- **Total correctness**: also asserts C terminates
- A3 targets partial correctness (it proves safety, not termination)
- Hoare logic is the mathematical framework behind program verification; all the barrier/invariant work is a specific way to find Hoare-style proofs automatically

93. A3's consecution in pictures



Consecution: yes!

- The barrier function B assigns a non-negative score to every safe state
- Consecution in barrier terms: executing one step cannot make B go from ≥ 0 to < 0
- Formally: $B(\sigma) \geq 0 \wedge T(\sigma, \sigma') \Rightarrow B(\sigma') \geq 0$
- This is exactly the SMT query A3 sends to Z3 for the consecution obligation

94. Fixpoint operators and Kleene iteration

- Computing R exactly requires the least fixpoint of $F(X) = S_0 \cup \text{post}(T, X)$
- **Kleene iteration**: start from $R_0 = S_0$, repeatedly apply F until no change
- Over finite domains: converges in finitely many steps
- Over infinite domains (e.g. integers): may never converge without widening
- **Widening** forces convergence by coarsening: replace an infinite ascending chain with a finite over-approximation
- Classic widening: widen intervals, convex hulls, etc.

95. Why barrier certificates avoid the fixpoint problem

- Kleene iteration requires *computing* the reachable set step by step
- Barrier certificate approach: directly *guess* (via template) a function B that satisfies all three obligations
- The solver then **verifies** the three obligations by checking satisfiability — no iteration needed
- This is called "assume–verify" or "certificate search": find a witness, then verify it
- Advantage: if the template is right, solver returns quickly
- Disadvantage: if no template of the given form works, the approach fails (returns UNKNOWN)

96. Template-based invariant synthesis

- A **template** is a parametric form for B, e.g. linear: $B(\mathbf{x}) = \mathbf{c}^\top \mathbf{x} + c_0$
- The unknowns are the coefficients $(c_1, \ldots, c_n, c_0)$
- The three obligations become constraints on the coefficients:
  - Initiation: for all $\sigma_0 \in S_0$, $\mathbf{c}^\top \sigma_0 + c_0 \geq \epsilon$
  - Consecution: for all (σ, σ') with $T(\sigma, \sigma')$, $\mathbf{c}^\top \sigma' + c_0 \geq 0$ if $\mathbf{c}^\top \sigma + c_0 \geq 0$
  - Separation: for all $\sigma \in U$, $\mathbf{c}^\top \sigma + c_0 \leq -\epsilon$
- Solving these constraints gives the certificate coefficients

97. Existential vs universal quantification in synthesis

- The synthesis problem is: $\exists c_1, \ldots, c_n : \ \forall \sigma$: obligations hold
- This is an $\exists\forall$ problem — harder than pure SMT (which handles $\exists$ over fixed formula)
- Strategy: use **Farkas' lemma** (for linear arithmetic) or **SOS certificates** (for polynomial) to convert the $\forall$ into a form Z3 can handle
- Result: the synthesis problem reduces to a single SMT query with unknown coefficients
- For polynomial templates, the SOS relaxation converts it to a semidefinite program (SDP) — covered next chapter

98. Summary: what eliminates a candidate

- A candidate (possible bug at location ℓ) is eliminated when:
  1. **Path-level UNSAT**: the path condition joining S_0 to ℓ with the unsafe predicate is UNSAT
  2. **Inductive invariant**: an I passing Init+Consec+Sep is found (barrier certificate is one way)
  3. **Guard discharge**: the guard protecting the bug makes $\Phi \wedge U$ UNSAT directly
- Methods 1 and 3 are the same (path-level check); Method 2 is the barrier/invariant approach for loop-sensitive proofs
- A3 tries the cheaper methods first, escalates to barrier only when needed

99. Chapter 5 summary: consecution vocab

Core concepts:

- Inductive invariant (I)
- Init / Consecution / Separation
- Hoare triple {P} C {Q}
- Induction metaphor for programs

Advanced:

- k-induction
- Invariant strengthening
- Template + coefficient search
- Kleene iteration and widening
- $\exists\forall$ synthesis vs SMT

One-sentence summary

Consecution is the inductive step: the safe zone must be closed under program transitions. Finding it automatically requires template-based synthesis and SMT.

100. Halfway checkpoint: what you now know

- **Ch 1**: bugs, safety, reachability, TP/FP, soundness
- **Ch 2**: SAT/UNSAT/UNKNOWN, Z3, SMT theories, Craig interpolants
- **Ch 3**: symbolic execution, path trees, path conditions, path explosion
- **Ch 4**: CFGs, state tuple, transition relation, loops, interprocedural
- **Ch 5**: inductive invariants, initiation, consecution, separation, k-induction

What's coming

Ch 6: barrier certificates and SOS polynomials (the math) Ch 7: CEGAR, IC3/PDR, interpolants Ch 8: concolic execution, synthesis, confidence, CI ratchet

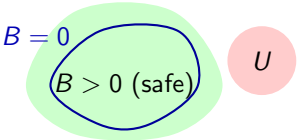
Chapter 6: Barrier Certificates and Sum-of-Squares

Chapter goal

Learn the central proof technique in A3: what a barrier certificate is, what the three obligations mean in polynomial language, and why sum-of-squares (SOS) makes the problem tractable.

101. The barrier intuition: a level-set fence

- A **barrier certificate** is a function $B : \mathbb{R}^n \to \mathbb{R}$ such that:
  1. $B(s) \geq \epsilon > 0$ for all initial states $s \in S_0$
  2. $B(s) \leq -\epsilon < 0$ for all unsafe states $s \in U$
  3. B does not decrease across transitions: $B(s) \geq 0 \wedge T(s, s') \Rightarrow B(s') \geq 0$
- The level set $\{s : B(s) = 0\}$ is a **barrier surface** separating safe from unsafe



102. Why a function (not just a set) is convenient

- An inductive invariant is a *set*. Searching over all sets is intractable.
- Representing the invariant as a **function** allows parametric search: choose a template $B(\mathbf{x}) = \sum_\alpha c_\alpha \mathbf{x}^\alpha$, unknown c_α
- The three obligations become constraints on the coefficients c_α
- Solve for $c_\alpha \Rightarrow$ you have the certificate
- **Polynomial** B is a natural choice: closed under addition and multiplication, broadly expressive, and SMT-tractable via SOS

103. The three polynomial obligations

Polynomial barrier obligations

For polynomial $B(\mathbf{x})$ and program transitions:

$$\begin{aligned} \text{(Init)} \quad & \forall s \in S_0 : B(s) \geq \epsilon \\ \text{(Consec)} \quad & \forall s, s' : B(s) \geq 0 \wedge T(s, s') \Rightarrow B(s') \geq 0 \\ \text{(Sep)} \quad & \forall s \in U : B(s) \leq -\epsilon \end{aligned}$$

- Each obligation is a *universal* statement over states
- Negating gives existential queries — Z3 can check these via SMT
- For polynomial B and polynomial constraints, **SOS** gives a systematic way to satisfy them

104. What is a polynomial?

- A **polynomial** in variables $x_1, \ldots, x_n$ is a finite sum of **monomials**:

$$p(\mathbf{x}) = \sum_{\alpha} c_\alpha \mathbf{x}^\alpha, \quad \mathbf{x}^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}$$

- Degree of a monomial: $\sum_i \alpha_i$
- Example ($n = 2$, degree 2): $B(x, y) = 3x^2 - 2xy + y^2 + x - 4$
- A3 typically uses degree-2 polynomial templates (fast) and escalates to degree 4 if needed
- More monomials = more expressive invariants, but harder to solve for coefficients

105. Sum-of-squares (SOS): what it means

Definition

A polynomial $p(\mathbf{x})$ is **sum-of-squares (SOS)** if it can be written as:

$$p(\mathbf{x}) = \sum_{i=1}^{k} q_i(\mathbf{x})^2$$

for polynomials q_i.

- An SOS polynomial is *always* non-negative: $p(\mathbf{x}) \geq 0$ for all $\mathbf{x}$
- NB: the converse is not always true (non-negative does not always imply SOS — this is a famous result)
- But for our purposes: SOS is a sufficient certificate of non-negativity, and it can be *checked efficiently*

106. SOS and semidefinite programming

- $p(\mathbf{x}) = \mathbf{m}^\top Q\,\mathbf{m}$ where $\mathbf{m}$ is the vector of monomials and Q is a matrix
- p is SOS if and only if Q is **positive semidefinite** (PSD): $Q \succeq 0$
- Checking or finding a PSD Q is a **semidefinite program (SDP)** — solvable in polynomial time
- This is the key: SOS reduces non-negativity (hard in general) to SDP (tractable)
- A3 uses an SDP solver (e.g. CVXPY/MOSEK) to find the certificate coefficients

107. Positivstellensatz: SOS in constrained domains

- We need non-negativity not over all $\mathbb{R}^n$ but only over the initial/unsafe sets (which are defined by polynomial inequalities)
- **Positivstellensatz** (Stengle, 1974): if $p > 0$ on the set $\{g_1 \geq 0, \ldots, g_k \geq 0\}$, then there exist SOS polynomials σ_i such that:

$$p = \sigma_0 + \sigma_1 g_1 + \sigma_2 g_2 + \ldots$$

- This converts the constrained non-negativity into an unconstrained SOS problem
- **In A3**: the initial set S_0 and unsafe set U are described by polynomial inequalities; Positivstellensatz makes the Init/Sep obligations solvable via SDP

108. The S-procedure shortcut

- For quadratic functions on linear constraints, there is a simpler tool: the **S-procedure**
- Given: want to prove $p(x) \geq 0$ whenever $g(x) \geq 0$ (where p, g are quadratic)
- S-procedure: this holds if there exist $\lambda \geq 0$ such that $p(x) - \lambda g(x)$ is SOS
- This is a single SDP feasibility problem: find λ and the SOS decomposition
- Many of A3's consecution proofs for simple programs reduce to an S-procedure SDP

109. Finding the barrier: the synthesis SDP

- Template: $B(\mathbf{x}) = \mathbf{c}^\top \phi(\mathbf{x})$ where ϕ is the vector of monomials up to degree d
- Obligations become linear matrix inequalities (LMIs) in the coefficients $\mathbf{c}$:
  - Init: $\mathbf{c}^\top \phi(\sigma_0) \geq \epsilon$ for each initial state generator
  - Consec: via Positivstellensatz, a matrix inequality in $\mathbf{c}$
  - Sep: $\mathbf{c}^\top \phi(u) \leq -\epsilon$ for each unsafe state generator
- Solve the SDP: if feasible, the solution $\mathbf{c}^*$ gives the barrier certificate
- If infeasible: no barrier of the chosen degree exists; try higher degree or another method

110. DSOS and SDSOS: cheaper variants

- **SOS** requires a full positive semidefinite matrix — $O(d^3)$ cost for degree d
- For large programs with many variables, this is prohibitively expensive
- **DSOS** (diagonally dominant SOS): restrict Q to be diagonally dominant (a sufficient condition for PSD that is a linear program, not SDP)
- **SDSOS** (scaled diagonally dominant SOS): slightly looser restriction, still LP-solvable
- DSOS/SDSOS are **sound subsets of SOS**: if DSOS finds a certificate, it is also valid under SOS
- Tradeoff: DSOS is faster but might miss certificates SOS would find

111. Why A3 uses multiple polynomial degrees

- Degree-1 (linear): cheapest; handles programs with linear data flow; misses many real invariants
- Degree-2 (quadratic): handles quadratic state relationships; covers most reliable-bug patterns
- Degree-4 (quartic): required for programs with multiplication of program variables
- A3 starts with degree-2 and escalates to degree-4 only if degree-2 returns infeasible
- Cost: number of monomials in n variables of degree $\leq d$ is $\binom{n+d}{d}$, which grows fast

112. Connecting barrier certificates to the A3 talk

- When you say "the kitchensink tries multiple solvers" in the talk, barrier certificates are **one of those solvers**
- Specifically: A3 has a "BarrierSolver" module that implements the SDP-based synthesis
- When a path-level UNSAT check fails (UNKNOWN or SAT), A3 tries to find a barrier certificate
- If barrier synthesis succeeds: FP eliminated with an inductive proof
- If barrier synthesis fails/unknown: escalate further (DSE, Houdini, etc.)
- The barrier solver is the "formal certification" layer — the most rigorous elimination method

113. Barrier certificate example: DIV_ZERO

- Program computes `result = total / count` at end of accumulation loop
- State space: $(total, count)$ where count is incremented each iteration
- Initial state: $(0, 0)$; loop increments count each iteration
- Unsafe: $count = 0$ at the division site
- **Barrier**: $B(total, count) = count - \frac{1}{2}$
  - Init: initially $count = 0$ — but the loop must have been entered before division is reached. Context: division only reachable after at least 1 iteration $\Rightarrow$ $count \geq 1$ at that point
  - Sep: $count = 0 \Rightarrow B = -\frac{1}{2} < 0$. ✓
  - Consec: $count \geq \frac{1}{2}$ plus the loop increment $\Rightarrow$ B stays ≥ 0. ✓

114. Limitations of barrier certificates

- **Discrete programs**: SOS is designed for continuous domains; discrete (integer) jumps require adaptation
- **Complex control flow**: exception edges, dynamic dispatch make the transition relation hard to encode as polynomial
- **Template choice**: if the right degree is not tried, the method returns infeasible incorrectly
- **Scalability**: for programs with 50+ symbolic variables, SDP becomes slow
- A3 addresses these by: encoding integers as bounded reals; abstracting complex control; trying multiple degrees; using DSOS for large programs

115. Prajna and Jadbabaie (2004): origin of barrier certificates

- Prajna & Jadbabaie introduced barrier certificates for continuous dynamical systems in 2004 (HSCC)
- Original setting: differential equations $\dot{x} = f(x)$; barrier function must satisfy $\dot{B} \geq 0$ wherever $B = 0$
- A3 adapts this to discrete programs: replace $\dot{B} \geq 0$ with $B(s') \geq 0$ given $B(s) \geq 0$
- The discrete version is more natural for programs; the key insight (level-set fence) transfers directly
- You can cite this as context: "barrier certificates were originally invented for control systems; we apply the same principle to Python programs"

116. Chapter 6 summary: barrier certificate vocab

Core concepts:

- Barrier function B
- Level set / barrier surface
- Init / Consec / Sep obligations
- Polynomial template + degree
- Monomials and coefficients

Algebraic machinery:

- SOS = sum of squares
- SDP = semidefinite program
- Positivstellensatz
- DSOS / SDSOS variants
- S-procedure for quadratics

Chapter 7: CEGAR, Abstraction, IC3/PDR, and Interpolants

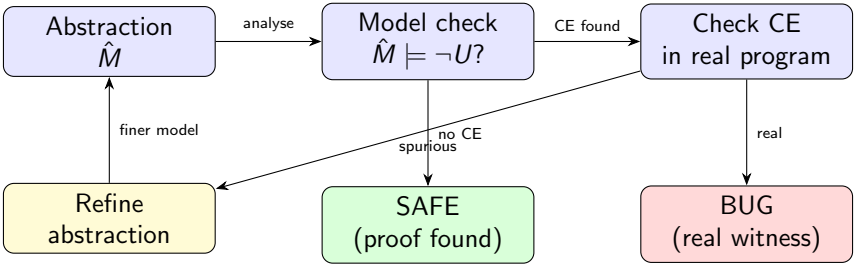
Chapter goal

Learn counterexample-guided abstraction refinement (CEGAR), the IC3/PDR incremental invariant builder, and Craig interpolants. These are the "repair loop" methods when simple proofs fail.

117. The abstraction-refinement idea

- **Problem**: full symbolic execution is too expensive; simplified models (abstractions) are cheap but imprecise
- **Observation**: if analysis on the *abstract* model says "no bug", the real program is also safe (if abstraction is sound over-approximation)
- **Problem**: the abstract model might say "bug possible" when it is only a phantom of the abstraction, not real
- **Solution**: use the spurious counterexample to *refine* the abstraction, then retry
- This cycle — abstract, analyse, refine — is **CEGAR**

118. CEGAR: counterexample-guided abstraction refinement



119. What makes a counterexample spurious?

- The abstract model $\hat{M}$ over-approximates the real program M
- A path in $\hat{M}$ leading to U may not correspond to any path in M
- **Spurious counterexample**: the abstract CE path has no real execution trace
- Detection: run the CE path symbolically on the real program; if the path condition becomes UNSAT before reaching U, it is spurious
- Spurious CE gives valuable information: the abstract path was infeasible *for a reason* — extract that reason as a new abstraction predicate

120. Predicate abstraction

- **Predicate abstraction**: abstract the program state by a finite set of predicates $\{p_1, \ldots, p_k\}$
- Abstract state: a boolean vector recording which predicates are true
- Example: predicates $\{x > 0, y \neq 0, z < 10\}$ — abstract state has $2^3 = 8$ elements
- Transition in the abstract model: which combinations of predicate truth values are reachable in one step?
- Starting with coarse predicates (e.g. just $\{\text{true}\}$): fast but imprecise
- After CEGAR refinement: predicates grow more specific; abstraction becomes tighter

121. How a spurious CE leads to a new predicate

- Spurious CE path: states $\sigma_0 \to \sigma_1 \to \ldots \to \sigma_k$ in abstract model, but infeasible in real model
- The infeasibility is witnessed by an UNSAT proof: $\Phi_0 \wedge T_1 \wedge \ldots \wedge T_k \wedge U$ is UNSAT
- **Interpolant extraction**: compute Craig interpolant sequence $I_1, \ldots, I_{k-1}$ from the UNSAT proof
- Each I_i is a predicate over the relevant variables at step i — it is a new "distinguishing fact"
- Add these predicates to the abstraction: the refined model *cannot* reproduce the spurious path
- Repeat until no spurious CEs remain or a real CE is found

122. Craig interpolants: the formal definition

Craig interpolant

Given $A \wedge B$ is UNSAT, the Craig interpolant I satisfies:

1. $A \models I$ (A implies I)
2. $I \wedge B$ is UNSAT (I is incompatible with B)
3. I uses only symbols common to A and B

- Intuition: I is the "minimal shared reason" why A rules out B
- In CEGAR: A = prefix of spurious path, B = suffix reaching unsafe; I = fact true after prefix that blocks the suffix
- Interpolant algorithms extract I from the UNSAT proof structure (proof of resolution)

123. CEGAR in the A3 context

- A3 does not run a full CEGAR loop as a standalone pass (that is ModelChecking-style)
- Instead, CEGAR principles appear in:
  1. **Template refinement**: if barrier certificate fails, use the failed-obligation model to extract new monomials to add to the template
  2. **DSE path feedback**: concolic execution finds new constraints; fed back to re-query solver with richer path condition
  3. **Houdini weakening**: start with strongest invariant candidate, weaken on counterexample
- When you mention CEGAR in the talk, this is the context: iterative refinement of what-we-know based on solver feedback

124. IC3/PDR: the modern verifier's algorithm

- **IC3** (Incremental Construction of Inductive Clausal Invariants; Bradley 2011) also called **PDR** (Property Directed Reachability)
- Instead of searching for a global invariant, IC3 incrementally builds a *frame sequence* $F_0, F_1, \ldots, F_k$ such that:
  - $F_0 = S_0$ (initial states)
  - Each $F_i \supseteq F_{i+1}$ (monotone tightening)
  - $F_k \cap U = \varnothing$ (safety of the last frame)
  - Each F_i is inductive relative to F_{i-1}
- When two consecutive frames converge ($F_k = F_{k+1}$), we have found an inductive invariant

125. IC3 frames as clause sets

- Each frame F_i is represented as a **conjunction of clauses** (like an CNF formula)
- A **clause** c in frame F_i asserts: "no state in F_i satisfies the negation of c"
- When IC3 finds a state s in F_i that *could* lead to U, it **blocks** s by adding a new clause to F_{i-1}
- The blocking clause is derived by generalising s into a cube (a conjunction of literals), then negating it
- This process propagates blocking obligations backward through frames: "what predecessors could lead here?"

126. IC3 algorithm sketch

1. Check if $S_0 \cap U \neq \varnothing$: if so, immediate bug at step 0
2. Check if a predecessor of U can be in S_0 (1-step reachability check)
3. If yes: counterexample found (or generalise and block)
4. If no: create new frame $F_{k+1} = \{U^c\}$ (starts as "not unsafe"); propagate clauses
5. For each clause in F_i: try to push (propagate) it to F_{i+1} by checking inductiveness
6. If a clause propagates to F_{k+1}: extend the frame sequence
7. If convergence: invariant found; if bug trace: real CE

- Key: each SAT call either finds a CE to block or proves a clause advances

127. PDR for software: CHC formulation

- IC3/PDR was originally for hardware (finite bit-vectors)
- For software: use **Constrained Horn Clauses (CHC)** as the formulation
- A CHC is a clause of the form: $B_1(\mathbf{x}_1) \wedge \ldots \wedge B_k(\mathbf{x}_k) \wedge \phi \Rightarrow H(\mathbf{y})$
- **Horn clauses** model recursive functions and loops naturally: the predicate symbol H plays the role of the loop invariant
- Solving a CHC system = finding interpretations for all uninterpreted predicates such that all clauses hold
- This is exactly the "find the inductive invariant" problem in Horn clause form

128. SeaHorn: CHC solving for programs

- **SeaHorn** is a verification framework that encodes C/C++ programs as CHC systems and solves them
- A3 adopts a similar CHC encoding for Python interprocedural analyses
- Each function's pre/postcondition becomes a Horn clause; recursive calls become head predicates
- A CHC solver (e.g. Spacer inside Z3) then finds the invariant interpretations
- Benefit: CHC solving separates the "program encoding" from the "invariant finding" — each can improve independently
- Spacer inside Z3 is a PDR-style CHC solver; A3 can invoke it directly

129. The SLAM project and predicate abstraction at scale

- **SLAM** (Microsoft, Ball & Rajamani, 2002) was the first large-scale CEGAR-based verifier
- Verified device driver property violations in Windows kernel
- Key technique: predicate abstraction + CEGAR, fully automated
- SLAM's insight: the right predicates come from the code itself (variable guards, comparisons at branches)
- A3's approach to predicate discovery follows the same heuristic: use branch conditions and comparison predicates from the function's own source

130. Lazy abstraction and impact

- **Lazy abstraction** (Henzinger et al., 2002): do not build the full abstract program upfront; refine *only the parts of the abstract model that were actually traversed by the CE path*
- Benefit: avoids refining parts of the program that are irrelevant to the current bug
- Result: dramatically fewer CEGAR iterations needed in practice
- BLAST (Berkeley Lazy Abstraction Software verification Tool) demonstrated this on real C programs
- A3 applies a similar "on-demand refinement" philosophy: only deepen analysis along paths that need it

131. Chapter 7a summary: CEGAR and interpolants

Core concepts:

- Abstraction / spurious CE
- CEGAR loop
- Predicate abstraction
- Interpolant = shared reason for UNSAT

Major systems:

- SLAM (Windows drivers)
- BLAST (lazy abstraction)
- IC3/PDR (frame-based)
- CHC + SeaHorn + Spacer

132. IC3 in the A3 portfolio: what role it plays

- A3's kitchensink portfolio includes an IC3/PDR-style invariant builder for cases where:
  - The barrier SDP is infeasible (no polynomial certificate exists at that degree)
  - The path condition is satisfiable but all simple guards fail
- IC3 builds clause-based invariants using the Z3 SAT/UNSAT feedback loop
- IC3 does not need a polynomial template — it infers the invariant structure from blocking cubes
- Limitation: works best on linear arithmetic; harder for nonlinear programs
- In practice: IC3 often handles the cases the SOS certificate misses, and vice versa

133. Interpolant-based template expansion

- When the barrier SDP is infeasible with monomials $\{1, x, y, x^2, xy, y^2\}$: maybe the right invariant needs term x^2y or some other monomial not in the template
- Strategy: run a CEGAR step; get the failed-consecution model (σ, σ')
- Compute interpolant between $\Phi_0(\sigma) \wedge T(\sigma, \sigma')$ and $\neg B(\sigma')$
- Interpolant uses a new predicate structure suggesting which new monomials to add to the template
- Add those monomials; re-run SDP — often finds the certificate on the second try
- This is the "CEGAR inside barrier synthesis" loop mentioned in slides 13f

134. k-induction and IC3: the relationship

- k-induction checks: given the last k states all satisfy P, does the next state satisfy P?
- IC3 generalises k-induction: frames $F_0, \ldots, F_k$ capture exactly what holds after at most k steps
- IC3 dynamically determines the right k — it is an **adaptive** induction depth finder
- When k-induction returns UNKNOWN at depth k, IC3 can continue to depth $k + 1$ by extending the frame sequence
- This makes IC3 more powerful than fixed k-induction configurations

135. Abstract interpretation: a brief overview

- **Abstract interpretation** (Cousot & Cousot, 1977) is the theoretical framework behind most static analysers
- Key idea: define a **Galois connection** between concrete and abstract domains
- Abstract domain examples:
  - **Interval domain**: variable x tracked as range $[l, u]$
  - **Octagon domain**: pairs of variables: $\pm x \pm y \leq c$
  - **Polyhedra domain**: conjunctions of linear inequalities
- Analysing a program in the abstract domain by propagating abstract values is fast but imprecise
- A3 uses interval-like approximations for variable ranges before escalating to SMT/barrier

136. Widening in abstract interpretation

- At a loop back-edge, abstract values grow with each iteration (e.g. interval $[0,1],[0,2],[0,3],\ldots \to [0,\infty)$)
- **Widening** operator∇: forces convergence by extrapolating to the limit
- Example: $[0,1]\nabla[0,3] = [0,+\infty]$ — widen the upper bound to infinity
- After widening: run **narrowing** to tighten the result: $[0,+\infty]\triangle[0,100] = [0,100]$
- Widening preserves soundness (over-approximation) but loses precision
- A3 uses abstract interpretation results as an initial over-approximation; SMT/barrier refine where needed

137. Connecting to the RISE talk: the kitchensink portfolio

- The A3 "kitchensink" is the ordered portfolio of methods tried for each candidate:
  1. **Guard check**: direct UNSAT on $\Phi \wedge U_t$ (cheapest)
  2. **Abstract interpretation**: interval / octagon over-approximation (fast)
  3. **Barrier certificate** (SOS/DSOS): degree-2 — inductive polynomial invariant
  4. **IC3/PDR clause learning**: clause-based invariant builder
  5. **k-induction**: $k \in \{1, 2, 4\}$ unrolling
  6. **CEGAR loop**: predicate abstraction + interpolant refinement
  7. **DSE**: concolic confirmation (see Chapter 8)
- Each step escalates cost; the system stops at first SAFE or UNSAFE conclusion

138. When every portfolio method returns UNKNOWN

- All portfolio methods exhausted; none proved safe or found a real bug
- Policy: retain candidate with confidence score = "LOW" (high uncertainty)
- Report in SARIF with evidence chain showing which methods were tried
- **Agentic triage**: a language model examines the SARIF, the code context, and assigns a final severity label based on contextual reasoning
- The triage agent looks for: Is this reachable in production context? Are there upstream guards? Is the code path exercised in tests?
- UNKNOWN survivors after triage become the "genuine uncertainty" report — not suppressed

139. Summary: the methods and what they prove

Method	**Proves**	**Cost**
Path UNSAT	No bug on one path	Cheapest
AI / Interval	Unreachable by range	Fast
Barrier (SOS)	Inductive poly invariant	Medium
IC3/PDR	Clause invariant	Medium
k-Induction	Property up to depth k	Scales with k
CEGAR	Predicate abstraction	Iterative
Concolic (DSE)	Confirms real witness	Medium

140. Chapter 7 summary: the refinement methods

CEGAR:

- Abstract $\rightarrow$ analyse $\rightarrow$ check CE $\rightarrow$ refine
- Predicate abstraction
- Interpolants from UNSAT proof
- Lazy: refine only what matters

IC3/PDR:

- Frame sequence $F_0, \ldots, F_k$
- Blocking cubes from SAT models
- Convergence = invariant
- CHC generalisation for software

Chapter 8: Concolic Execution and DSE

Chapter goal

Learn what "concolic" means, how it combines concrete and symbolic execution to confirm or refute bug witnesses, and how A3 uses it as the final gate before reporting a true positive.

141. What “concolic” means

- **Concolic** = **con**crete + **sym**bolic execution, running simultaneously
- Coined for the DART tool (Godefroid, Klarlund, Sen, 2005)
- **Concrete pass**: run the program with actual input values — fast, no approximation
- **Symbolic pass**: simultaneously track symbolic expressions alongside concrete values
- At branches: the concrete value determines which branch to take; the symbolic side records the constraint
- Result: one complete concrete trace paired with its exact path condition

142. Why concolic avoids path explosion

- Pure symbolic execution forks at every branch: exponential paths
- Concolic only follows **one path** at a time (the concrete execution determines the path)
- After one run, **negate one branch condition** to get a new input that takes a different path
- Repeat: explore the unexplored branches systematically (like a guided DFS over the path tree)
- This is **dynamic symbolic execution (DSE)**: symbolic + driven by concrete runs
- Key advantage: no spurious paths — every explored path is a real execution

143. Concolic execution step by step

1. Start with some concrete input (e.g. $x = 0, y = 1$) and symbolic variables (α, β)
2. Execute concretely: $x = 0$ takes the false branch of `if x > 0`
3. Record path condition: $\Phi = (\alpha \leq 0)$
4. **Negate last constraint**: $\alpha > 0$ — ask solver for model: $x = 1$
5. Re-run with $x = 1$: now takes the true branch; new path condition $\Phi = (\alpha > 0)$
6. Continue negating branches to explore the full path tree systematically

- Each iteration discovers a new path; solver generates the input that triggers it

144. Concolic as witness confirmation in A3

- When SMT returns SAT with model $(x = v_1, y = v_2, \ldots)$, A3 knows: *symbolically*, the bug is reachable
- But: due to over-approximation, the symbolic model might not correspond to a real execution
- A3 seeds the concolic engine with the SAT model as the starting concrete input
- Concolic runs the program concretely with those values
- If the bug is hit: **real TP** — report with the concrete trace as evidence
- If the bug is not hit (path diverges): the SAT model was spurious; discard, try next portfolio step

145. The concolic trace as evidence

- A confirmed concolic trace is maximally actionable evidence for a developer:
  - Exact input values that trigger the bug
  - Stack trace showing which function calls led to the bug site
  - Variable values at each frame
- A3 encodes this evidence in the SARIF record under the `codeFlows` field
- A developer can reproduce the bug by simply running the program with the provided inputs
- This sharply reduces the "I can't reproduce it" back-and-forth between security team and developer

146. Path divergence and why it happens

- **Path divergence**: the concrete execution takes a different branch than the symbolic model predicted
- Causes:
  - **Imprecise modelling**: symbolic model over-approximated a library call differently than reality
  - **Environment effects**: file system, time, random number generator — not modelled symbolically
  - **Type mismatches**: Python dynamic typing means an object can be a different type at runtime vs the assumption made symbolically
- When divergence occurs: the concrete and symbolic states differ; A3 detects this and flags the candidate as "inconclusive" (UNKNOWN)

147. DSE coverage: exploring new paths

- Concolic with path condition negation is the heart of **directed test generation**
- Starting from one seed input, systematically generate inputs covering all feasible branches
- This is used in A3 not just for confirmation but also for **exploration**: find new candidates in untested paths
- KLEE (LLVM-based concolic tool) showed this approach achieves dramatically higher branch coverage than random testing
- A3's concolic stage has a coverage budget: explore up to k path branches before stopping

148. Concolic and the false-negative risk

- Concolic only explores the paths it is guided to: it *cannot* prove absence of bugs on unexamined paths
- This is why concolic is used for **confirmation** (under-approximation), not for **elimination** (which needs over-approximation)
- Mixing them up: if A3 only used concolic and reported only concolic-confirmed bugs, it would miss bugs no path exploration reached (FN)
- Correct design: use over-approximate methods for elimination; use concolic only for TP confirmation

149. DART, KLEE, SAGE: the concolic ecosystem

- **DART** (2005, Bell Labs): first concolic tool; showed systematic test generation from random seeds
- **KLEE** (2008, Stanford): LLVM-based; tested 452 GNU utilities; found new bugs in many
- **SAGE** (2008, Microsoft): applied concolic to Windows file parsers; found hundreds of security bugs
- These tools validated that concolic at scale is practical; A3's concolic engine implements the same core ideas adapted to Python bytecode rather than native code

150. Chapter 8 summary: concolic vocabulary

Core concepts:

- Concolic = concrete + symbolic
- DSE (dynamic symbolic execution)
- Path condition negation
- Seed input from SAT model

A3 uses concolic for:

- TP confirmation (not FP elimination)
- Path divergence detection
- Concrete trace as SARIF evidence
- Coverage improvement

Chapter 9: Invariant Synthesis (Houdini, ICE, SyGuS)

Chapter goal

Learn the automated methods for synthesising loop invariants — Houdini, ICE learning, and SyGuS — which complement the barrier approach.

151. Houdini: assume strongest, weaken on CE

- **Houdini** (Flanagan & Leino, 2001): given a set of candidate predicates $\{p_1, \ldots, p_k\}$, find the largest *inductive* subset
- Algorithm:
  1. Start with conjunct $I = p_1 \wedge p_2 \wedge \ldots \wedge p_k$
  2. Check consecution of I: is there a state satisfying I that, after one step, violates some p_i?
  3. If yes: remove p_i from I; repeat
  4. If no: I is inductive — done
- Terminates because the set only shrinks (removes predicates, never adds)
- Result: the **maximal inductive subset** of the candidates

152. ICE learning: invariants from examples

- **ICE** (Implication Counter-Example learning; Garg et al., 2014): frames invariant learning as a learning problem with three kinds of examples:
  - **Positive** examples: reachable states (must be in invariant)
  - **Negative** examples: unsafe states (must be out of invariant)
  - **Implication** examples: pairs (s, s') where $s \to s'$ (consecution examples)
- A **learner** (e.g. decision tree or linear classifier) finds a classifier consistent with all examples
- A **teacher** (SMT solver) generates new examples when the learner's invariant candidate fails
- Interplay: teacher and learner alternate until no new counterexamples appear

153. SyGuS: syntax-guided synthesis

- **SyGuS** (Syntax-Guided Synthesis): synthesise a program expression that meets a formal specification, restricted to a given **grammar** of allowed expressions
- For invariant synthesis: the grammar specifies which terms are allowed (e.g. linear combinations of program variables)
- The synthesis engine searches the grammar for a term satisfying the spec (e.g. inductive invariant obligations)
- Benefit: integrates user knowledge ("the invariant has this shape") without requiring exact coefficients
- SyGuS competition benchmarks include invariant synthesis; A3 can invoke SyGuS-compatible solvers

154. How synthesis methods relate to barriers in A3

- **Barrier SDP**: searches for a polynomial B with real-valued coefficients via SDP — best for safety/numeric invariants
- **Houdini**: searches for a conjunction of given predicates — best when good predicates are already known (from contracts, annotations)
- **ICE**: learns arbitrary classifiers — best for complex invariants that don't fit polynomial templates
- **SyGuS**: follows a grammar — best when invariant structure is known but coefficients are not
- A3 kitchensink uses all four: Houdini for contract-based checks; ICE for residual cases; SyGuS for structured repairs; SOS for continuous safety

155. Chapter 9 summary: synthesis methods

- **Houdini**: weaken from maximum conjunct; terminates; best with good candidate predicates
- **ICE**: learn from positive/negative/implication examples; flexible classifier shape
- **SyGuS**: grammar-constrained synthesis; integrates domain knowledge
- All three are **complete up to the expressiveness of the template/grammar/classifier class**
- All three use an SMT oracle (teacher or verifier) to generate counterexamples — Z3 again

Chapter 10: Confidence Scoring, Triage, and CI Ratchet

Chapter goal

Understand how A3 assigns confidence scores, how the agentic triage layer works, what the CI ratchet policy is, and how all of this fits into a continuous integration workflow.

156. Why raw static analysis output is noisy

- A static analyser flagging 10,000 findings per run is useless: developers ignore everything
- **Signal-to-noise problem**: real bugs are rare; false positives dominate
- A3's solution: multi-layer filtering with attached evidence, scored by confidence
- **Confidence** is a scalar in $[0, 1]$ reflecting: how certain are we this is a real, exploitable bug?
- Confidence feeds into: developer priority queue, agentic triage routing, CI ratchet thresholding

157. Evidence types and confidence contributions

- Each piece of evidence shifts confidence:
  - **UNSAT proof**: eliminates candidate entirely (confidence of being a bug: near 0)
  - **SAT + concolic confirmed**: confidence near 1 — real, reproducible bug
  - **SAT, no concolic confirmation**: confidence 0.6–0.8
  - **UNKNOWN, all methods**: confidence 0.3–0.5
  - **Taint source confirmed**: high confidence for injection bugs
  - **Dead code path**: low confidence (path likely not exercised)
- **Stochastic risk model**: Bayesian-style combination of evidence weights

158. Confidence intervals and interval overlap

- Each confidence estimate comes with an **interval**: $[\text{low}, \text{high}]$ reflecting uncertainty about the estimate itself
- Example: SAT + near-concolic: confidence interval $[0.72, 0.91]$
- When comparing two findings: use interval overlap to assess statistical distinguishability
- If intervals overlap significantly: the two findings may be equally risky; do not over-rank
- A3 reports the interval in SARIF so CI tooling can apply appropriate thresholds

159. The agentic triage layer

- After static + concolic stages, surviving candidates go to an **agentic triage** step
- A large language model (LLM) agent receives:
  - The SARIF record with evidence chain
  - The source code context (function + caller + callee)
  - The symbolic path condition in human-readable form
  - Historical data: has this location been flagged before? Was it a FP?
- The agent outputs: severity label, rationale, recommended action (fix, ignore, investigate)
- Human reviewer sees the triage output: dramatically less cognitive load than raw SARIF

160. Agentic triage vs hallucination risk

- Risk: the LLM agent might hallucinate a rationale that sounds plausible but is wrong
- Mitigations in A3:
  1. The agent is **grounded**: it can only reference evidence present in the SARIF record
  2. The agent's reasoning is **logged**: every claim is traceable to a specific evidence item
  3. A **self-consistency check**: the agent's conclusion is validated against the solver outcome
  4. If agent disagrees with solver proof: solver wins (proofs beat opinions)
- The agent is the "final interpreter", not the "final authority" — formal evidence always overrides

161. The CI ratchet: preventing regression

- **Ratchet metaphor**: a ratchet mechanism can tighten but never loosen
- In CI: once a finding is eliminated (proved FP), never re-report it as a finding
- Once a bug is fixed (TP resolved), never regress to having that bug
- **Implementation**: a persistent finding database (SARIF store) in the CI pipeline
- Each new run diffs against the previous state: only *new* findings (regression) or newly resolved findings (improvement) are surfaced

162. The ratchet policy in practice

- **New TP found**: CI fails; developer must fix before merge
- **Previously known TP**: CI status tracks it; escalated if unfixed for too long
- **Previously eliminated FP re-appears**: treated as a regression — something changed that invalidated the proof; CI investigates
- **New UNKNOWN**: CI flags for human review but does not block merge
- This policy creates a **monotonically improving** security posture: the codebase can only get safer over time

163. SARIF as the integration format

- Every output from A3 is a SARIF file: structured JSON with standardised fields
- Key SARIF fields used by A3:
  - `ruleId`: bug type (DIV_ZERO, SQL_INJECTION, etc.)
  - `level`: error / warning / note
  - `properties.confidence`: the numerical confidence score + interval
  - `codeFlows`: concolic trace (if confirmed)
  - `relatedLocations`: solver evidence chain
- SARIF is understood by GitHub Advanced Security, VS Code, and most IDE security plugins
- A3's SARIF output plugs directly into these tools without adapters

164. How A3 outputs are consumed by developers

- **Pull request check**: A3 runs on every PR; new high-confidence findings block merge
- **IDE annotation**: SARIF displayed inline in editor (red underlines with evidence hover)
- **Security dashboard**: aggregate view of finding counts, trend, by category
- **Chatbot interface**: developer can ask "why is this flagged?" and the agent explains using the evidence chain
- The goal: verification proofs and concolic witnesses are as accessible as unit test failures

165. The episode arc in the talk: what each episode delivered

- **Episode I** (security): End-to-end pipeline for taint-based security bugs; SQLi, CMDi, PathTraversal
- **Episode II** (reliability): Extended to numeric reliability bugs; DIV_ZERO, OOB, NULL, UNCAUGHT
- **Episode III** (scale & CI): Kitchensink portfolio; confidence scoring; SARIF; CI ratchet
- Each episode's "deliverable" is the key innovation slide: the pipeline architecture, the 67-bug-type checklist, the CI ratchet policy
- When you say "by the end of Episode III": all pieces are in place; it's a production-grade pipeline

166. What "Z3-builder level operational usefulness" means

- This phrase from the project origin refers to the target bar: the system must be useful enough that **Nikolaj Bjørner** (a domain expert who builds the underlying solver) would actually rely on it
- It is not "impressive demo"; it is "daily practice tool"
- Criteria for that bar:
  - False positive rate below human tolerance for alarm fatigue
  - Evidence quality good enough to act on without further investigation
  - CI integration that fits naturally into existing development workflow
  - Scales to real-world project sizes (not just toy examples)
- The CI ratchet and confidence intervals are direct responses to this bar

Chapter 11: A3 Full Picture and Presentation Tips

Chapter goal

Connect every concept from this primer back to specific moments in the A3 walkthrough. Walk through the complete pipeline as a confident presenter.

167. The A3 pipeline: complete diagram

- **Discovery**: 67+ bug type checkers flag candidates
- **Symbolic execution**: build $\sigma = (\mathrm{pc}, \rho, h, \tau, \kappa, e, \Phi)$ and Z3 queries
- **Kitchensink**: guard check $\rightarrow$ AI $\rightarrow$ barrier $\rightarrow$ IC3 $\rightarrow$ CEGAR
- **DSE**: confirm TPs concretely
- **Triage**: LLM assigns severity with grounded reasoning
- **CI**: ratchet policy enforces monotonic improvement

168. Slide 1: deliverable framing — what to say

- The talk opens with the **deliverable**: a working system that finds real security and reliability bugs with formal elimination of false positives
- What to emphasise:
  - Not a prototype, not a paper claim — a system that produced SARIF records for real code
  - The key differentiator: evidence-backed results (proofs or witnesses), not just heuristic suspicion
- Avoid: "we formulate the problem as..." — that is the wrong register for RISE; lead with what ships
- Vocabulary you now know: SARIF, TP/FP, barrier certificate, SMT proof

169. Slide 11: state tuple — what to say

- When presenting $\sigma = (\mathrm{pc}, \rho, h, \tau, \kappa, e, \Phi)$:
  - pc: "which line of bytecode is next"
  - ρ: "what each variable symbolically equals"
  - h: "the heap — objects and their fields"
  - τ: "what type each object actually is"
  - κ: "taint level — did this value come from a user?"
  - e: "any pending exception currently propagating"
  - Φ: "the accumulated constraints that got us here"
- Say: "this 7-tuple is the full world-state from which the verifier reasons"

170. Slide 13b: barrier obligations — what to say

- When you show the three obligations:
  1. "Initiation: when the function starts, we're already inside the safe zone"
  2. "Consecution: every step of the program keeps us inside the safe zone" — stress: this is what we check for every loop iteration
  3. "Separation: the safe zone never overlaps the bug condition"
- The "fence" metaphor works well for RISE: "we build a mathematical fence around safe states; no execution can jump over it"
- If someone asks "what if the fence can't be built?": say "we escalate to IC3/PDR which builds a clause-based version"

171. Slides 13c–13g: the 20-paper map — what to say

- These slides show how A3 connects to 20 foundational papers across 5 groups
- You don't need to explain every paper — pick 1–2 per group and explain the connection
- Group by narrative:
  - "Stengle and Parrilo give us the math" (SOS, Positivstellensatz)
  - "IC3/Bradley and SeaHorn give us the algorithm" (clause-based, CHC)
  - "SLAM and BLAST give us the engineering" (industrial-scale CEGAR)
  - "Houdini and ICE give us the synthesis" (learning invariants)
  - "DART/KLEE give us the witnesses" (concolic confirmation)

172. Handling expert questions: 5 common questions at RISE

1. **"Why polynomial certificates and not just abstract interpretation?"**
   — AI widening is imprecise; barriers give exact proofs with checker certificates
2. **"How do you handle unknown Python library calls?"**
   — Conservative abstraction: treat return as unconstrained symbolic value; UNKNOWN routing
3. **"What is your FP rate on open-source projects?"**
   — Reference the evaluation section; quote the observed FP reduction percentage
4. **"How does your SOS approach handle integers (discrete)?"**
   — Integer intervals embedded in reals; integer assertions discharged by linear arithmetic first
5. **"Does this scale to 100k LOC?"**
   — Modular: analyse per-function; interprocedural via summaries; CI run on incremental changes

173. Terminology cheat sheet: what each term means in a sentence

- **Barrier certificate**: "a polynomial function that acts as a mathematical fence, proved to exclude the unsafe states from the reachable set"
- **Consecution**: "the mathematical condition ensuring the program cannot escape the safe zone in one step"
- **Concolic execution**: "running the program with concrete inputs while simultaneously tracking symbolic expressions, to confirm bugs without path explosion"
- **SOS**: "a way of proving a polynomial is always non-negative by expressing it as a sum of squares—the computationally tractable certificate form"
- **CHC**: "Horn clauses encoding program transitions—the language in which we ask Z3 to find a loop invariant"

174. Terminology cheat sheet (continued)

- **CEGAR**: "iteratively refine an abstract program model using the counterexamples that expose its imprecision"
- **IC3/PDR**: "incrementally build a sequence of tightening invariant frames until two frames are identical—that's your inductive proof"
- **Interpolant**: "the minimal shared reason why two parts of a program formula are contradictory—used to extract new predicates for CEGAR refinement"
- **CI ratchet**: "a policy that allows the security posture to only improve: once a bug is fixed, it can never reappear without breaking the build"
- **Kitchensink**: "the ordered portfolio of verification methods tried for each candidate—cheapest first, escalating until a decision is reached"

175. Pre-talk confidence: the two-sentence summary of each method

- **Path condition check**: find the constraints the program must satisfy to reach the bug; if they are contradictory (UNSAT), the bug is impossible.
- **Barrier certificate**: find a polynomial that stays positive in safe states and negative in unsafe ones; prove that program transitions can never push it from positive to negative (consecution).
- **CEGAR**: abstract the program coarsely; if the abstract model shows a bug, check if it is real; if not, learn why and tighten the abstraction.
- **IC3/PDR**: block bad states one by one, propagating blocking clauses backward through frames until the frames stabilise into a full invariant.
- **Concolic**: start with a concrete input from the solver; run the program while tracking symbolic constraints; confirm the bug is hit, or detect that the path diverges.

Chapter 12: Comprehensive Glossary

How to use this chapter

These final slides define every term that appears in the A3 walkthrough. Use them for quick reference before or during the talk.

176. Glossary: A

Abstract interpretation Analysis technique that approximates program semantics over an abstract domain (intervals, polyhedra, etc.) rather than concrete values.

Abstraction A simplified model of a program; over-approximate abstractions are sound.

Alert fatigue Developer response of ignoring all alerts because too many are false positives.

Agentic triage An LLM-agent layer that contextualises static analysis findings before reporting to developers.

177. Glossary: B–C

Barrier certificate	A function B satisfying init, consecution, and separation obligations, certifying program safety.
Basic block	Maximal straight-line code sequence with single entry and exit.
CEGAR	Counterexample-guided abstraction refinement: iterative abstraction tightening.
CFG	Control flow graph — nodes are basic blocks, edges are possible control transfers.
CHC	Constrained Horn Clauses — logical encoding of program transitions used in CHC solvers.
CI ratchet	Continuous-integration policy allowing only security improvement, never regression.
Clause	A disjunction of literals; the atomic unit of SAT/IC3 invariant construction.
Concolic	Concrete + symbolic execution running simultaneously; DART/KLEE style.
Consecution	The inductive step of a safety proof: the safe zone is closed under transitions.
Craig interpolant	A formula derived from an UNSAT proof, used to extract new predicates.

178. Glossary: D–I

DSOS Diagonally dominant SOS: a linear-programming relaxation of SOS certificates.

DSE Dynamic symbolic execution; a.k.a. concolic testing.

False negative (FN) A real bug that was missed by the analyser.

False positive (FP) A flagged candidate that is not a real bug; eliminated by proof.

Houdini Algorithm that finds the maximal inductive conjunction of given candidate predicates.

ICE learning Invariant learning via implication counterexamples; combines examples and SMT.

IC3/PDR Frame-based invariant builder; property directed reachability.

Inductive invariant A set I satisfying initiation, consecution, and separation.

Interpolant See *Craig interpolant*.

179. Glossary: K–P

k-induction Inductive proof using the last k states as the hypothesis.

Kitchensink A3's ordered portfolio of verification methods tried for each candidate.

Liveness property A property asserting something good eventually happens (not targeted by A3).

Path condition Φ Conjunction of branch constraints accumulated during symbolic execution.

PDR Property directed reachability; same algorithm as IC3.

Positivstellensatz Algebraic certificate theorem connecting polynomial positivity to SOS decompositions over semialgebraic sets.

Predicate abstraction Abstract state represented as a boolean vector of predicate truth values.

180. Glossary: R–Z

Reachability The question: "can the program reach a bad state?"

Safety property A property asserting nothing bad ever happens; modelled as unsafe set U.

SARIF Static Analysis Results Interchange Format; structured JSON for analysis findings.

SAT Satisfiable: a formula has at least one satisfying assignment.

SDP Semidefinite program; the optimisation problem arising from SOS certificate synthesis.

SMT Satisfiability modulo theories; SAT extended with arithmetic, arrays, etc.

Soundness If the analyser says safe, it is safe (no false negatives).

SOS Sum of squares; certificate of polynomial non-negativity enabling SDP synthesis.

State tuple σ A3's 7-tuple $(\mathrm{pc}, \rho, h, \tau, \kappa, e, \Phi)$ characterising all analysis state.

Taint analysis Tracking whether data flows from untrusted sources to dangerous sinks.

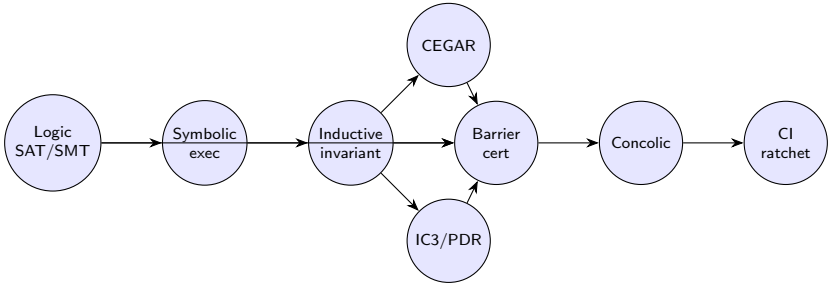
UNSAT Unsatisfiable: no assignment makes the formula true; used as FP-elimination proof.

Z3 Microsoft's open-source SMT solver; primary decision procedure in A3.

181. Key equations at a glance

Safety goal:	$R \cap U = \varnothing$
Inductive invariant:	$S_0 \subseteq I,\ I$ closed under $T,\ I \cap U = \varnothing$
Barrier obligations:	$\forall s \in S_0 : B(s) \geq \epsilon,\ \forall (s, s') \in T : B(s) \geq 0 \Rightarrow B(s') \geq 0,\ \forall s \in U : B(s) \leq -\epsilon$
Consecution check:	$I(s) \wedge T(s, s') \wedge \neg I(s')$ is UNSAT
FP elimination:	$\Phi(\sigma) \wedge U_t(\sigma)$ is UNSAT
TP witness:	$\Phi(\sigma) \wedge U_t(\sigma)$ is SAT, model confirmed by concolic

182. Dependency map: which ideas build on which



- Logic and symbolic execution are prerequisites for everything
- Inductive invariants are the central concept; barrier, CEGAR, and IC3 are all implementations

183. The 10 questions to ask yourself before the talk

1. Can I explain consecution in one sentence without notes?
2. Can I say why UNSAT eliminates a false positive?
3. Can I describe barrier certificates as a "level-set fence"?
4. Can I explain what SAT + concolic confirmation means?
5. Can I sketch the kitchensink portfolio order?
6. Can I say what the CI ratchet policy does?
7. Can I handle the question "why not just test more?"?
8. Can I explain why UNKNOWN is handled conservatively?
9. Can I connect IC3/PDR to the frame-based invariant idea?
10. Can I say what Z3 returns and what each result means?

184. What you *don't* need to memorise

- The exact paper proofs of Positivstellensatz or Stengle's theorem
- The internal algorithm of Z3's Spacer CHC solver
- The precise SDP formulation for degree-4 barrier certificates
- The Hoare logic inference rules for compound statements
- The precise definition of the Galois connection in abstract interpretation

Rule of thumb

For every concept: know the **one-sentence intuition** and the **consequence for A3**. You do not need to be able to implement it from scratch.

185. The "so what" for each method

- **SMT/Z3 UNSAT**: "so what" — false positives are eliminated with a mathematical proof
- **Barrier certificates**: "so what" — loops are handled rigorously; not just unrolled
- **CEGAR**: "so what" — when simple proofs fail, we iteratively improve the model until one succeeds or a real bug is confirmed
- **IC3/PDR**: "so what" — an alternative invariant search that works even when polynomial templates fail
- **Concolic**: "so what" — surviving candidates that look like real bugs are confirmed with a concrete input you can hand to a developer
- **CI ratchet**: "so what" — this is not a one-time audit; it's a continuously enforced safety monotonicity property

186. Practice: describe the pipeline to a non-expert

Your elevator pitch

"We run Python code through a tool that symbolically explores all possible inputs simultaneously. When it finds a potential crash or security hole, it either mathematically proves the problem can't actually happen (in which case we discard it), or it finds the exact input that triggers it (in which case we report it with a reproducible test case). The whole thing runs in CI so the codebase can only get safer with each commit."

- This covers: symbolic execution, barrier elimination, concolic confirmation, CI ratchet
- No technical jargon needed for this version
- For RISE: expand each phrase with the technical detail from this primer

187. Practice: describe the pipeline to a verification expert

Expert pitch

"We encode Python semantics as a symbolic transition system over a 7-tuple state. Candidates are discharged by SMT UNSAT (path-level) or inductive barrier certificates (loop-level) synthesised via SOS/SDP. Residual unknowns go through an IC3-style clause learner and CEGAR predicate refinement. TP survivors are confirmed concolicly via negation of SAT model constraints. Findings are reported as SARIF with attached Positivstellensatz certificates or concolic traces; the CI ratchet enforces monotonic security improvement."

188. Common misconceptions to correct

- "**Symbolic execution always causes path explosion**" — No: A3 manages this with loop invariants and bounded unrolling; concolic avoids it by following one path
- "**SMT can always give a definitive answer**" — No: UNKNOWN is a valid output; undecidable fragments exist
- "**Formal verification means no false positives and no false negatives**" — No: A3 is sound-ish (aims for low FN) but not perfect; explicit UNKNOWN handling
- "**Barrier certificates require the program to have polynomial data flow**" — Partially true: nonlinear programs need higher-degree templates; complex heap uses conservative abstractions

189. The connection to control theory (for control engineers in RISE)

- Barrier certificates were invented in control theory (Prajna & Jadbabaie, 2004) for continuous dynamical systems
- The discrete program version mirrors the continuous version: replace $\dot{B}(x) \geq 0$ with $B(x_{k+1}) \geq 0$ if $B(x_k) \geq 0$
- A RISE audience may include control engineers who know Lyapunov functions — barrier certs are their safety analog
- Lyapunov functions prove **stability** (convergence to set); barrier certificates prove **safety** (avoidance of bad set)
- Key distinction: Lyapunov decreases globally; barrier only needs to stay non-negative, not decrease

190. The connection to formal methods (for FM researchers in RISE)

- RISE may include formal methods researchers who know model checking, Alloy, TLA+
- A3 is an **automated program verifier** (like SLAM, CBMC, CPAchecker) for Python
- Key distinctions from classical model checking:
  - Works on real source code, not specifications
  - Uses polynomial certificates (not just SAT-based BDD model checking)
  - Targets bug *categories* (67+ types) rather than one property at a time
  - Pairs proofs with concolic witnesses for actionability
- If asked about completeness: A3 is not complete (UNKNOWN outcomes exist); the goal is high-confidence coverage, not exhaustive proof

191. The confidence interval question

- RISE: "What does a confidence interval of [0.7, 0.9] mean operationally?"
- It means: the Bayesian posterior weight on "this is an exploitable bug in production" is between 0.7 and 0.9
- The width of the interval reflects uncertainty about the model (how well our symbolic semantics mirrors the runtime)
- A narrow interval [0.85, 0.90] = highly calibrated estimate; a wide one [0.2, 0.8] = high uncertainty, needs more investigation
- The CI ratchet threshold is configurable: by default, block on confidence ¿ 0.75; surface for review on confidence ¿ 0.5

192. Things the talk does NOT need to cover

- Full proof of soundness of the symbolic semantics (too long; refer to technical report)
- Every SDP solver parameter and tuning option (irrelevant to RISE audience)
- The implementation language and testing framework for A3 itself (Python + pytest)
- Precise performance benchmarks on every BugsInPy project (have the numbers ready but don't recite them)
- Comparison to every commercial SAST tool (stick to scientific comparators: Infer, CBMC, CPAchecker)

193. Last-minute memory devices

- **Barrier = fence**: three nails — Init (post in ground), Consec (fence is solid), Sep (fence excludes danger zone)
- **CEGAR = ABCs**: Abstract, Bug check, refine if Counterexample is Spurious
- **IC3 = tightening rings**: each frame is a tighter ring around the reachable states; when two rings coincide, you have the invariant
- **Concolic = GPS**: start from solver directions, drive concretely, check if you arrive at the bug
- **UNSAT = alibi**: the formula has no witness to the crime — the bug is provably impossible

194. Quick quiz: test yourself

1. What are the three barrier certificate obligations? *(Init, Consecution, Separation)*
2. What does Z3 return when a bug is provably unreachable? *(UNSAT)*
3. What is a spurious counterexample? *(A CE in the abstract model with no real execution trace)*
4. What does "concolic" stand for? *(concrete + symbolic)*
5. What does the CI ratchet prevent? *(Security regression: only improvement allowed)*
6. What is consecution? *(The inductive step: safe zone closed under transitions)*
7. What is SOS used for in A3? *(Certifying barrier certificate non-negativity via SDP)*
8. What does UNKNOWN mean as a solver result? *(Undecided; handled conservatively by retaining candidate)*

195. The three sentences that tie the whole talk together

Opening hook

"Every Python project ships with invisible bugs. We built a system that can prove they don't exist — or prove they do, with a reproducible trace."

Middle bridge (after Episode II)

"What unifies all the methods is one idea: find a mathematical fence around the safe states and prove that no execution can cross it. Barrier certificates are one way to build that fence; IC3/PDR and CEGAR are others."

Closing claim

"The CI ratchet turns this from a one-time scan into a monotonicity property: every commit either maintains or improves the security posture, backed by formal evidence."

196. The most important slide in the deck (13b)

- Slide 13b shows the three barrier obligations as the unifying interface
- **This is the intellectual core of the talk**: every method in the kitchensink is trying to satisfy these three conditions
- Practise explaining this slide until it feels natural:
  1. Point at "Init" — say: "initial states are inside the fence"
  2. Point at "Consec" — say: "every step keeps us inside the fence"
  3. Point at "Sep" — say: "the fence never overlaps the unsafe zone"
  4. Pause, then say: "if we can find a certificate B satisfying all three, the bug is provably impossible"
- If you nail slide 13b, the rest of the talk follows naturally

197. Appendix: key papers by theme

Control-theory origin	Prajna & Jadbabaie 2004 (HSCC), Prajna 2007 (TAC)
SOS foundations	Shor 1987, Parrilo 2000 (thesis), Lasserre 2001, Stengle 1974
Tractable relaxations	Ahmadi/DSOS 2019, Waki sparsity, Ames/CBF
Clause-based invariants	Bradley IC3 2011, Een/PDR 2011, SeaHorn/Gurfinkel 2015
Predicate abstraction	Ball/SLAM 2002, Henzinger/BLAST 2002, McMillan interpolants 2003
Invariant synthesis	Flanagan/Houdini 2001, Garg/ICE 2014, Alur/SyGuS 2013
Concolic testing	Godefroid/DART 2005, Cadar/KLEE 2008, Godefroid/SAGE 2012

198. Appendix: A3 modules and their techniques

Module	Technique	Chapter
`SymbolicExecutor`	Symbolic execution + Z3	Ch 3, 4
`BarrierSolver`	SOS/SDP barrier synthesis	Ch 6
`IC3Solver`	IC3/PDR clause learning	Ch 7
`CEGARLoop`	Predicate abstraction + interpolants	Ch 7
`DSEEngine`	Concolic DSE confirmation	Ch 8
`HoudiniSolver`	Houdini max-inductive conjunct	Ch 9
`ConfidenceModel`	Bayesian evidence combination	Ch 10
`TriageAgent`	LLM-based contextual triage	Ch 10
`CIRatchet`	SARIF diff + CI enforcement	Ch 10

199. Final checklist before the talk

- ☐ I can say "consecution" confidently and explain it in one sentence
- ☐ I can explain why UNSAT = false positive elimination
- ☐ I can describe barrier certificates as a "mathematical fence"
- ☐ I can sketch the kitchensink portfolio order (5 methods)
- ☐ I can explain the CI ratchet in one sentence
- ☐ I know what SARIF is and why it matters
- ☐ I have answers ready for the 5 common RISE questions (slide 172)
- ☐ I have practised slides 13b (barrier obligations) and 167 (full pipeline)
- ☐ I can connect to control theory if a controls person asks
- ☐ I have the opening hook memorised (slide 195)

200. You are ready

What you now know

- Ch 1–2: Verification, safety, reachability, logic, SMT/Z3
- Ch 3–4: Symbolic execution, CFGs, state tuple, transition systems
- Ch 5: Inductive invariants, consecution, k-induction
- Ch 6: Barrier certificates, SOS, SDP, DSOS
- Ch 7: CEGAR, IC3/PDR, interpolants, abstract interpretation
- Ch 8: Concolic execution, DSE, witness confirmation
- Ch 9: Invariant synthesis — Houdini, ICE, SyGuS
- Ch 10–11: Confidence scoring, triage, CI ratchet, full pipeline
- Ch 12: Glossary and quick reference

Good luck with the presentation.